Vol autonome d'un drone Tello à travers un cerceau

Identification, asservissement, détection par réseau de neurones et architecture temps réel

Adrien Chaudron & **Samuel Stef** · Projet du cours *Contrôle de robots mobiles* · Septembre 2026
Code source : github.com/AdrienChaudron/drone-tello-cerceaux

1. Introduction

Ce projet a été réalisé dans le cadre du cours *Contrôle de robots mobiles*, enseigné par le professeur Hugues Garnier. Le cours ne comporte pas de cours magistraux : il est organisé en séances de TP aboutissant à un projet.

Nous avons choisi le sujet portant sur les drones DJI Tello, qui réunit identification de systèmes, asservissement, vision par ordinateur et informatique temps réel.

1.1 Code source

Le code source du projet est disponible sur GitHub : github.com/AdrienChaudron/drone-tello-cerceaux.

1.2 Dans la continuité d'un projet plus long

Le sujet portant sur les drones Tello était ouvert. Théo Rutschke, le doctorant en charge de notre groupe, nous avait fourni une archive de 2 Go pour travailler sur la plateforme.

Dans cette archive, il y avait une étude de Georges Ghazi et Julien Voyer de l'ÉTS Montréal décortiquant l'asservissement du drone en hauteur ; mais il y avait surtout le rapport de TP de nos prédécesseurs Bastien Spyri et Evan Rolin. Nous nous sommes appuyés sur leur travail pour fixer notre objectif : faire passer le drone automatiquement dans un cerceau, sans pilotage.



Leur rapport nous a également évité de réexplorer plusieurs hypothèses sur l'acquisition des données du drone ; nous y revenons dans les parties concernées. Un support pour le cerceau a été fabriqué au département technique de Polytech Nancy. Cet objectif n'avait pas pu être atteint par nos prédécesseurs, faute de temps.

2. Asservir la hauteur et le lacet

Après une étude des ressources mises à notre disposition, le travail a été réparti en deux. Adrien Chaudron s'est chargé du réseau de neurones, en lien avec son double cursus à la FST Nancy en IA et vision pour la robotique ; Samuel Stef s'est chargé de l'asservissement et de l'informatique industrielle, en lien avec son double cursus à la FST Nancy en automatique (EEA).

Cette partie présente les premières étapes de l'asservissement. Le drone est piloté avec la bibliothèque `djitellopy` :

```
from djitellopy import Tello
tello = Tello()
```

2.1 La hauteur

Le premier axe qu'on a cherché à asservir fut la hauteur. On voulait faire le lien entre la commande que l'on peut envoyer avec la fonction python ci-dessous et la hauteur produite :

```
tello.send_rc_control(
    #...
)
```

Une boucle d'essai envoie une séquence de commandes adaptée à l'identification du système.

Identification du système

Notre système en hauteur se présente sous la forme suivante. Nous avons une entrée u(t) qui est un pourcentage de vitesse en hauteur entre -100 et +100, et en sortie, on récupère la distance drone - sol mesurée par le capteur.



Après essais, on estime le système comme étant un premier ordre avec intégrateur :

$$G(s) = \frac{K}{s(1+\tau s)} e^{-T_d s}$$

Avec les valeurs :

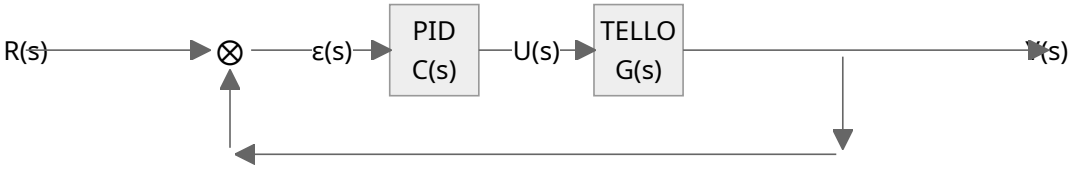
- K = 0.6, le gain statique
- τ = 0.3, la constante de temps
- T_d = 0.3, le retard du système

Le modèle a été estimé à partir des réponses mesurées, sans recourir à des outils d'identification robuste comme la CONTSID ou la *System Identification Toolbox* (modèles ARX). Il possède un pôle intégrateur en 0 et un pôle stable en $-1/\tau \approx -3{,}3$.

Le système est ensuite bouclé pour asservir la hauteur.

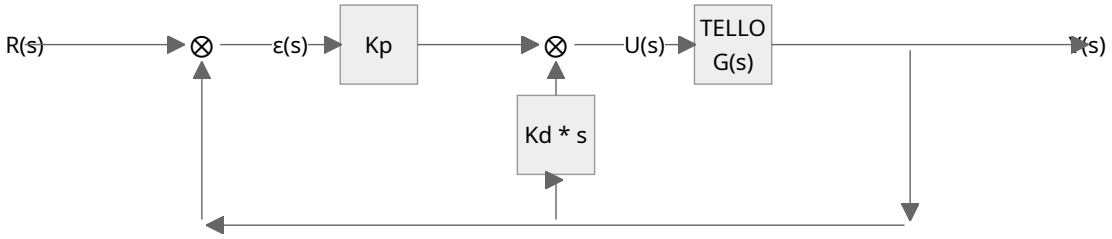
Système bouclé

Nous partons d'une structure de correction classique, le PID :



Notre correcteur sera un PD. Nous nous sommes basés sur le rapport de nos prédécesseurs, mais nous avons tout de même fait des essais avec un P et un PI.

Particularité, on va mettre la correction en dérivée sur la sortie et non sur l'erreur, et ce pour éviter que le système bouclé ne soit instable à cause de l'intégration sur la fonction de transfert du Tello :



Cela nous renvoie à une fonction de transfert sous cette forme (si on omet le retard) :

$$F_{CL}(s) = \frac{Y(s)}{R(s)} = \frac{\frac{KK_p}{\tau}}{s^2 + \frac{1+KK_d}{\tau}s + \frac{KK_p}{\tau}}$$

Qu'on vient comparer à la fonction de référence la plus proche :

$$F_{ref}(s) = \frac{\omega_0^2}{s^2 + 2\xi\omega_0 s + \omega_0^2}$$

La méthodologie est classique : on se fixe un cahier des charges en dépassement et en temps de montée. Faute de connaissance préalable du comportement du système bouclé, ces spécifications ont été ajustées par essais successifs. Les spécifications retenues sont :

- D_1 = 0.10, le dépassement
- t_m = 2.4 s, le temps de montée

Or :

$$\xi = \sqrt{\frac{ln(D_1)^2}{\pi^2 + ln(D_1)^2}} \quad ; \quad \omega_0 = \frac{\pi}{t_m\sqrt{1-\xi^2}}$$

À partir de là, on retrouve K_p et K_d :

```
D1 = 0.10;
tm = 2.4;

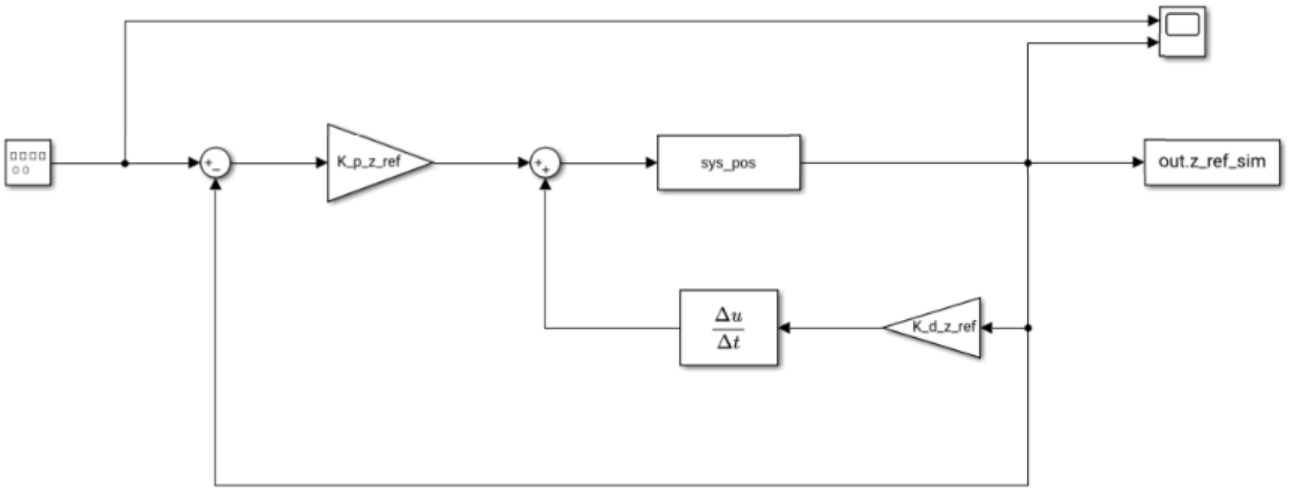
xi_ref = sqrt(log(D1)^2 / (pi^2 + log(D1)^2));
w_0_ref = pi / (tm * sqrt(1 - xi_ref^2));

% PD type
K_p_z_ref = (tau * w_0_ref^2) / K;
K_d_z_ref = (1 - 2 * xi_ref * sqrt(tau * w_0_ref * K_p_z_ref)) / K;
K_i_z_ref = 0;
```

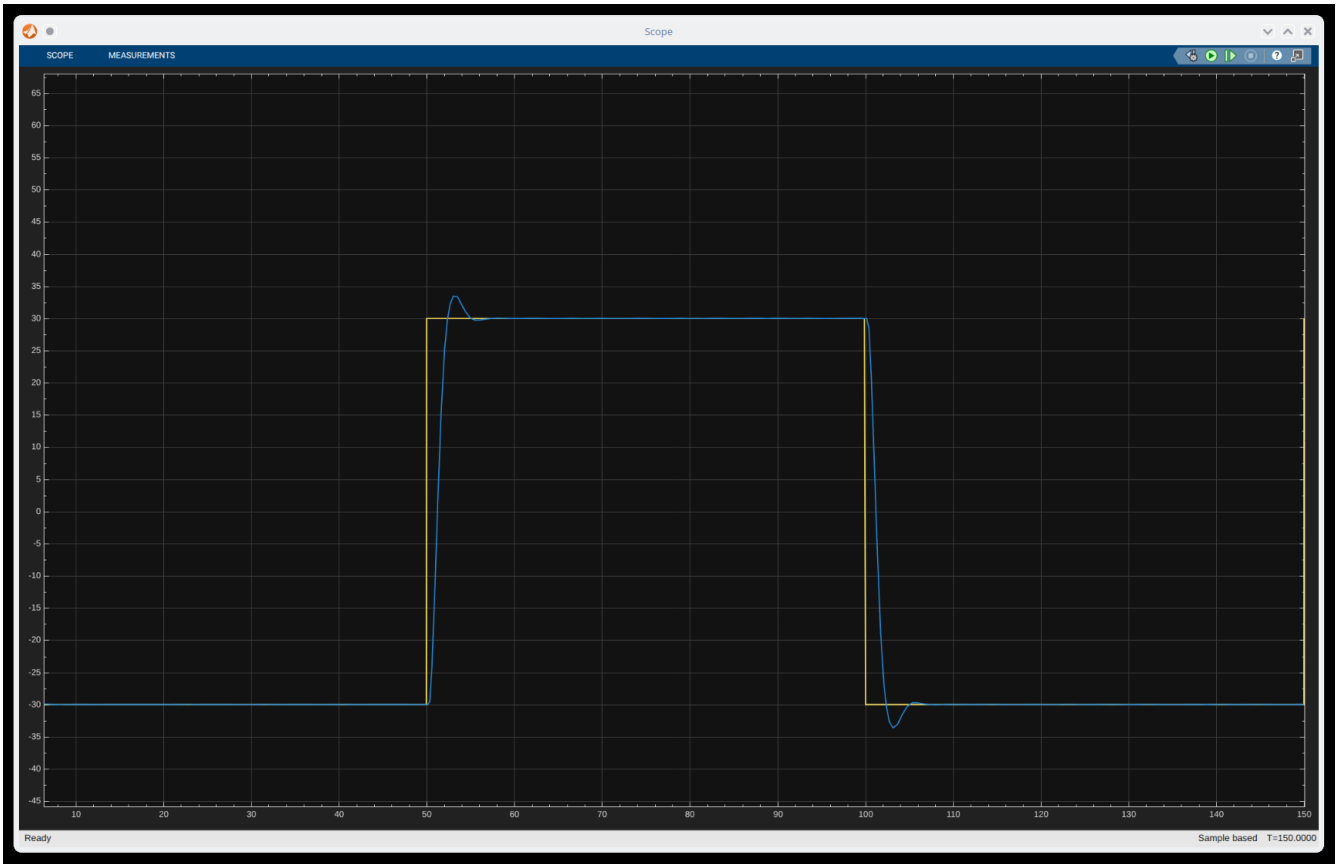
Résultats :

```
============================================================
REFERENCE MODEL PARAMETERS
============================================================
Damping ratio (m_ref) : 0.5912
Nat. frequency (w_0_ref): 1.6229 rad/s
------------------------------------------------------------
Proportional Gain (Kp) : 1.3170
Derivative Gain (Kd) : 0.0888
Integral Gain (Ki) : 0.0000
============================================================
```

Le correcteur est validé en simulation sous Simulink :



Ce qui nous donne le signal suivant dans le scope :



Les performances obtenues restent modestes : elles sont rapidement limitées par le matériel du drone. Des gains plus élevés rendent le système instable. Une identification plus robuste permettrait d'affiner le réglage du correcteur.

La loi de commande est implémentée en Python :

```
def get_command(K_p, K_d, reference, current_value, previous_value, Ts):
    err_t = reference - current_value
    derivation = (current_value - previous_value) / Ts
    command = K_p * err_t - K_d * derivation
    return command
```

2.2 Le lacet

L'asservissement du lacet (yaw en anglais) est sensiblement le même.

Identification du système

On a identifié le système de la même manière.



Et obtenu le même type de système, un premier ordre avec intégrateur :

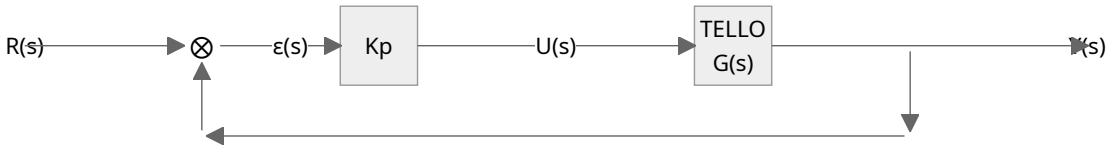
$$G(s) = \frac{K}{s(1+\tau s)} e^{-T_d s}$$

Avec les valeurs :

- K = 0.95
- τ = 0.3
- T_d = 0.2

Système bouclé

Pour le lacet, un correcteur proportionnel (P) s'est révélé suffisant. Le système bouclé a la forme suivante :



Les calculs pour le correcteur sont donc différents. Nous trouvons la fonction de transfert en boucle fermée suivante :

$$F_{CL}(s) = \frac{Y(s)}{R(s)} = \frac{\frac{KK_p}{\tau}}{s^2 + \frac{1}{\tau}s + \frac{KK_p}{\tau}}$$

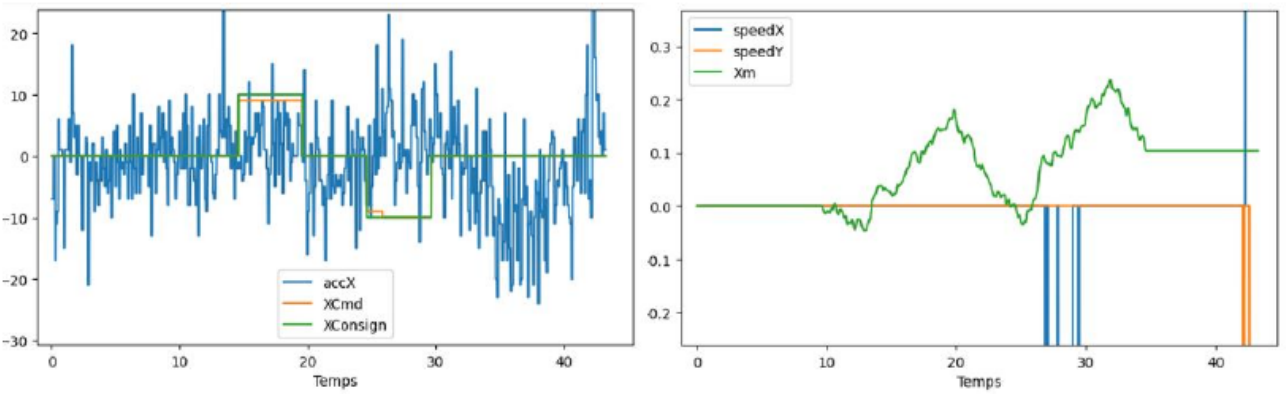
Nous sommes arrivés à la conclusion que les meilleures spécifications allaient être :

- D_1 = 0.25, le dépassement
- t_m = 1.3 s, le temps de montée

Ce qui nous donne K_p = 2.0 pour avoir un système bouclé stable.

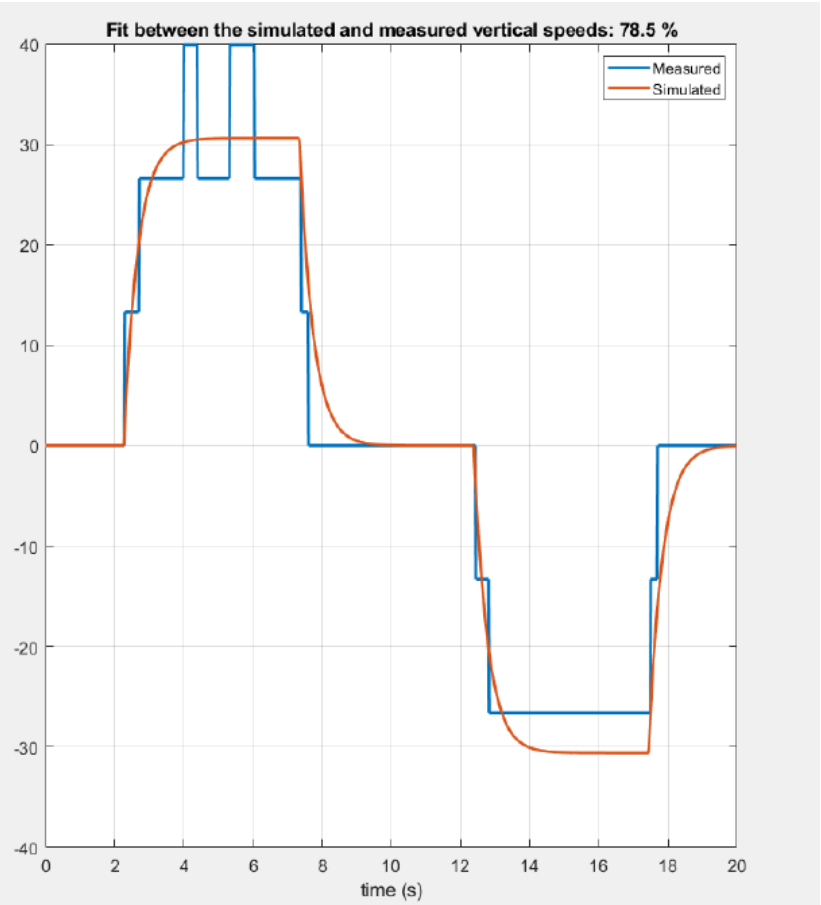
2.3 L'impossibilité de piloter par coordonnées

Pour cette partie, nous nous sommes basés entièrement sur les travaux de nos prédécesseurs. Ils ont essayé de passer par l'accéléromètre interne au drone mais les résultats n'étaient pas satisfaisants en terme de mesures :



Comme on peut le voir, le problème principal vient du fait que la seule mesure que l'on possède pour x et y sont les vitesses sur chaque axe, et que ces vitesses n'ont pas de plages de valeurs suffisamment élevées pour que les mesures aient un quelconque intérêt face aux bruits de mesures.

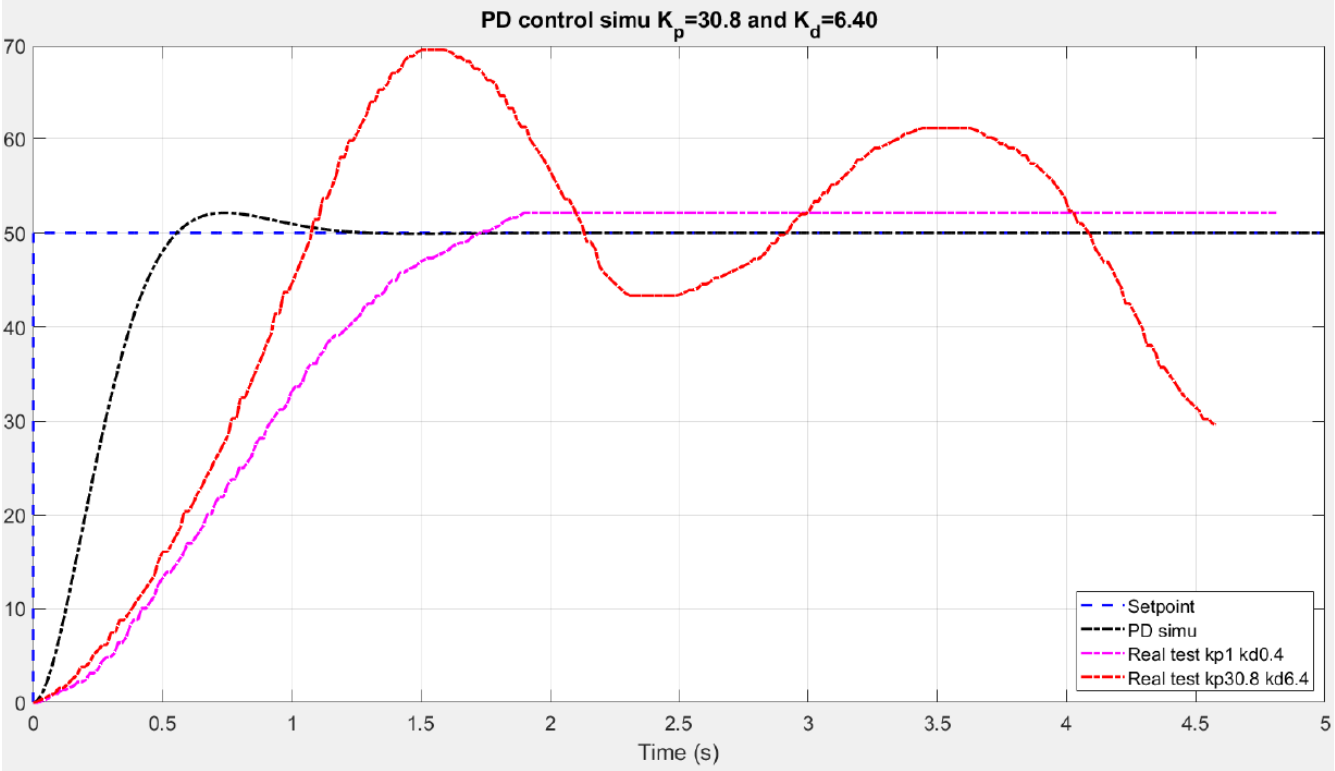
Ça se voit d'ailleurs sur le modèle de premier ordre qu'ils ont essayé de fit :



L'amplitude des valeurs est trop basse, et ils l'ont bien spécifié :

> *Nous avons donc des résultats corrects avec un FIT de 90% même si nous pouvons noter que le calcul interne à de grandes incrémentations et donc une faible précision, cela est amplifié par le coefficient de télémétrie.*

Conséquemment, la correction, bien que probablement bien designée, restait plutôt mauvaise :



3. Créer un réseau de neurones

Afin que le drone puisse naviguer à travers les cerceaux, il nous faut une architecture de vision : pour cela nous avons opté pour un réseau de neurones convolutif (CNN) de segmentation, qui localise le cerceau bien plus précisément qu'une régression directe ; sa robustesse aux conditions d'environnement (éclairage, bruit, couleur) vient, elle, des augmentations appliquées à l'entraînement. Afin de déterminer le meilleur réseau, nous avons entraîné plusieurs modèles avec des augmentations différentes et les avons comparés grâce aux métriques (précision, rappel, F1...), jusqu'à obtenir un modèle adapté à notre situation.

3.1 Pourquoi la segmentation ?

Un détecteur doit répondre à deux questions : y a-t-il un cerceau ? (classification) et où est-il exactement ? (localisation). En effet lors de nos premiers modèles nous nous sommes contentés uniquement de la régression ce qui avait pour effet de "perdre" la position du cerceau dans l'image.

3.2 Les données

Jeu de données et découpage

615 images de la caméra du Tello (960×720), prises dans le couloir et dans la cage : 374 avec cerceau, 241 vides. Règle d'annotation : si moins d'un tiers du cerceau est visible, l'image est vide. Chaque cerceau est annoté par une ellipse (centre, grand axe, petit axe, inclinaison), qui peut déborder de l'image. Nous faisons ainsi un découpage : 495 images d'entraînement et 120 de validation. Les images sont regroupées par tranches de 5 secondes, et chaque tranche va entièrement dans l'entraînement ou dans la validation : deux images quasi identiques, prises à une

fraction de seconde d'écart, ne peuvent donc pas se retrouver des deux côtés.

Prétraitement

- redimensionnement en 480×360
- passage de BGR (OpenCV) à RGB, valeurs ramenées dans [0, 1] puis normalisées avec la moyenne et l'écart-type d'ImageNet. Cette conversion est la même à l'entraînement, à la validation et en vol ;
- tenseur (canaux, hauteur, largeur) : un lot de 16 images est un tenseur (16, 3, 360, 480).

Le masque cible

Un cerceau vu par la caméra apparaît comme une ellipse. C'est elle qui est annotée sur chaque image (en pixels, image 960×720), puis prédite par le réseau. Elle est décrite par cinq paramètres :

Paramètre	Signification	Utilité
(cx, cy)	centre de l'ellipse	position du cerceau dans l'image
A	demi-grand axe	taille apparente, donc la **distance**
B	demi-petit axe (B ≤ A)	aplatissement
θ	inclinaison du grand axe	direction dans laquelle le cerceau est vu de biais

Le rapport B/A traduit l'angle de vue : 1 pour un cerceau vu de face, de plus en plus petit vu de biais (B/A ≈ cos de l'angle de vue).

Les métriques de localisation comparent ces paramètres à l'annotation : erreur du centre (en pixels), erreur relative du grand axe ($|A \text{ prédit} - A \text{ annoté}| / A \text{ annoté}$, qui se reporte directement sur la distance), erreur du rapport B/A (orientation) et erreur d'inclinaison θ (sur les ellipses aplaties, B/A < 0,85). Le « grand axe médian » est la médiane de l'erreur relative du grand axe sur la validation.

Le réseau apprend à reproduire un masque de l'anneau, dessiné automatiquement à partir de l'ellipse annotée : l'ellipse est ramenée à la résolution du masque (240×180, soit un pas de 2) et tracée avec une épaisseur de 4 % du diamètre au minimum 2 pixels. Une image vide a un masque entièrement noir ; une "corde" (arc trop court) a un masque inconnu, ignoré par la perte.

Image + ellipse annotée



Masque cible 240×180 (tube 4 % du diamètre)



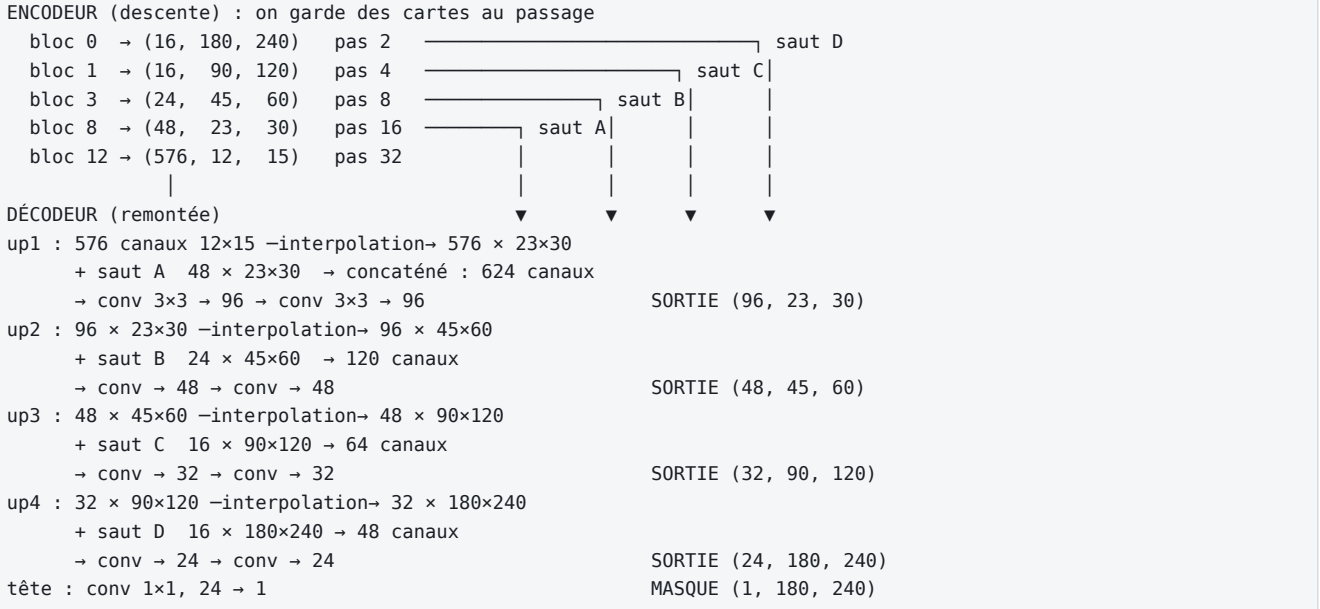
Augmentations (entrainement seulement)

Transformation	Probabilité	Paramètres	Rôle
Miroir horizontal	0,5	—	Cerceau vu de l'autre côté
Rotation de caméra H = K R K^{-1}	0,6	lacet ±12°, tangage ±8°, roulis ±12° ; K de la calibration	Le drone tourne ; ellipse transformée exactement
Zoom + décalage	0,5	échelle 0,6 à 1,3 ; décalage ±15 %	Distance et position variées
Visibilité minimale	—	au moins 34 % du contour dans l'image	Cohérent avec la règle du tiers
Couleur / lumière	0,8	luminosité ±35 %, contraste ±30 %, saturation ±40 %, teinte ±4	Contre-jour, exposition
Teinte tournée	0,5	teinte décalée d'une valeur tirée sur tout le cercle (0 à 360°)	Cerceau vert, bleu, jaune… : la couleur ne renseigne plus
Canaux permutés	0,3	ordre B, G, R tiré au hasard	Autre façon de changer la couleur
Niveaux de gris	0,2	image convertie en gris (3 canaux identiques)	Reconnaître le cerceau à sa forme seule
Flou de bougé	0,3	noyau ≤ 11 px	Mouvement du drone
Bruit	0,3	écart-type ≤ 8	Capteur
Rectangle masqué	0,3	côté ≤ 15 % de l'image	Occultation
Compression JPEG	0,5	qualité 35 à 80	Flux Wi-Fi
Collage sur un autre fond	0	désactivé	Créait des anneaux « fantômes »

Chaque image n'est vue qu'une fois par époque avec un tirage différent à chaque époque.

3.3 Le réseau RingSegNet

Le réseau a la forme d'un U : un encodeur réduit progressivement l'image en cartes de caractéristiques de plus en plus abstraites et de moins en moins résolues; un décodeur remonte la résolution en réutilisant les cartes intermédiaires de l'encodeur pour retrouver le détail fin du tube plastique.



L'encodeur : MobileNetV3-small

C'est un réseau conçu pour les téléphones : peu de calculs, mais pré-entrainé sur 1.2 million d'images (ImageNet), il sait déjà reconnaître bords, textures et formes. Ses 13 blocs sont faits de :

- Une convolution d'entrée 3*3 de pas de 2 (strides) (16 filtres), suivie d'une normalisation par lot et de l'activation hard-swish : c'est une approximation rapide de la *swish*, x · σ(x). La sigmoïde σ y est remplacée par une fonction affine écrêtée :

$$\text{h-swish}(x) = x \cdot \frac{\text{ReLU6}(x+3)}{6}$$

Elle vaut 0 pour x ≤ −3 et x pour x ≥ 3, avec une transition douce entre les deux. Elle ne demande aucune exponentielle et se quantifie bien en 8 bits, avec une précision quasi identique à la swish. MobileNetV3 la réserve à la première convolution et aux blocs profonds : les premiers blocs, qui travaillent sur de grandes cartes, gardent la ReLU, moins coûteuse.

- Une convolution 1x1 qui augmente le nombre de canaux, une convolution en profondeur 3x3 ou 5x5, un module squeeze-and-excitation : Chaque canal est résumé par sa moyenne. Une petite couche dense suivie d'une hard-sigmoïde en déduit un poids entre 0 et 1 par canal. Les canaux sont ensuite multipliés par ce poids : les plus utiles sont renforcés, les autres atténués.
- Une convolution finale 1*1 vers 576 canaux (hard-swish). La résolution est divisée par 2 cinq fois, toujours par des convolutions de strides de 2.

Bloc de l'encodeur	Sortie (canaux, H, L)	Pas cumulé	Rôle
bloc 0	(16, 180, 240)	2	**saut D** → up4
bloc 1	(16, 90, 120)	4	**saut C** → up3
bloc 3	(24, 45, 60)	8	**saut B** → up2
bloc 8	(48, 23, 30)	16	**saut A** → up1
bloc 12	(576, 12, 15)	32	entrée du décodeur (et de la tête de présence)

Le décodeur U-net

Il reconstruit progressivement une carte détaillée, à la moitié de la résolution d'entrée (180×240), à partir des petites cartes de l'encodeur. À chaque étape (UpBlock), il :

- agrandit la carte (interpolation bilinéaire) jusqu'à la taille de la carte de l'encodeur au même niveau ;
- colle les deux cartes l'une à l'autre (concaténation), pour retrouver les détails fins perdus pendant l'encodage ;
- affine le résultat avec deux convolutions 3×3, chacune suivie d'une normalisation et d'une ReLU.

Enfin, une convolution 1×1 donne pour chaque pixel un score (*logit*) : plus il est élevé, plus le pixel a de chances d'appartenir à l'objet recherché.

Bloc	Entrée (du bloc précédent)	Interpolation ×2	+ saut de l'encodeur	Après concaténation	Convolutions 3×3	Sortie
up1	576 × 12×15	576 × 23×30	+ saut A : 48 × 23×30	624 canaux	624 → 96 → 96	**(96, 23, 30)**
up2	96 × 23×30	96 × 45×60	+ saut B : 24 × 45×60	120 canaux	120 → 48 → 48	**(48, 45, 60)**
up3	48 × 45×60	48 × 90×120	+ saut C : 16 × 90×120	64 canaux	64 → 32 → 32	**(32, 90, 120)**
up4	32 × 90×120	32 × 180×240	+ saut D : 16 × 180×240	48 canaux	48 → 24 → 24	**(24, 180, 240)**
tête du masque	24 × 180×240	—	—	—	conv 1×1 : 24 → 1	**(1, 180, 240)** = masque

La tête de présence (classification)

La tête de présence répond à la question « y a-t-il un cerceau ? ». Elle part de la carte la plus profonde de l'encodeur (576 canaux de 12×15) : chaque canal est résumé par sa moyenne (moyenne globale), ce qui donne 576 valeurs décrivant toute l'image, indépendamment de la position. Après un Dropout de 0,2, une couche linéaire les combine en un score, que la sigmoïde transforme en probabilité p. Si $p < 0{,}5$, l'image est déclarée vide et aucune ellipse n'est calculée ; sinon, on passe à la localisation.

Résumé des opérations

Le réseau ne fait pas que des convolutions : à chaque étage, une opération précise répond à un besoin précis, que ce soit réduire l'image pour en comprendre le contenu, la remonter pour retrouver la position exacte du tube (morceaux de cerceau), décider si un cerceau est présent ou rendre l'apprentissage stable. Le tableau ci-dessous résume où intervient chacune de ces opérations et à quoi elle sert.

Où	Opération	Rôle
Encodeur (descente)	Convolutions de pas 2	Réduire la résolution, agrandir le champ réceptif
Modules squeeze-and-excitation	Moyenne globale par canal	Pondérer les canaux
Décodeur (remontée)	Interpolation bilinéaire + connexions de saut	Retrouver le détail fin
Tête de présence	Moyenne globale + Dropout 0,2	Décision globale, limiter le surapprentissage
Partout	Normalisation par lot	Stabiliser l'apprentissage

3.4 La perte : comment note-t-on le réseau

Pendant l'entraînement, le réseau fait une prédiction sur chaque image, et on la compare à la bonne réponse (l'annotation). La perte est une note d'erreur : plus elle est grande, plus le réseau se trompe. L'entraînement consiste à modifier les poids du réseau, petit à petit, pour faire baisser cette note.

Comme le réseau répond à deux questions, la perte additionne trois termes :

```
perte = erreur sur la présence        (y a-t-il un cerceau ?)
      + erreur pixel par pixel         (chaque pixel est-il du tube plastique ou du fond ?)
      + erreur de recouvrement (Dice)  (le masque prédit recouvre-t-il le vrai anneau ?)
```

L'erreur sur la présence

Pour chaque image, le réseau donne une probabilité p qu'il y ait un cerceau. On la compare à la vérité (1 = cerceau, 0 = vide) avec l'entropie croisée binaire (BCE) : l'erreur est faible quand le réseau est sûr et a raison, et très forte quand il est sûr et a tort (par exemple p = 0,99 sur une image vide). Elle est calculée sur toutes les images.

L'erreur pixel par pixel, et le piège du fond

On applique la même entropie croisée à chacun des 43 200 pixels du masque : le réseau a-t-il bien dit « tube » sur le tube et « fond » sur le fond ?

Le piège : l'anneau est fin, il ne couvre qu'environ 1 % des pixels. Un réseau paresseux qui répondrait « fond » partout aurait donc juste sur 99 % des pixels, avec une erreur déjà très faible, sans jamais trouver le cerceau.

Pour l'en empêcher, une erreur sur un pixel du tube compte 10 fois plus qu'une erreur sur un pixel du fond. Rater le tube devient alors coûteux, et le réseau doit absolument le trouver.

Les images vides participent aussi : leur masque attendu est entièrement noir, ce qui apprend au réseau à ne rien colorier quand il n'y a pas de cerceau.

L'erreur de recouvrement (Dice)

La perte de Dice mesure directement le recouvrement entre masque prédit P et masque vrai T (les annotations):

$\text{Dice} = 1 - (2 \cdot \Sigma\, P \cdot T + 1) / (\Sigma\, P + \Sigma\, T + 1)$

- **Recouvrement parfait** : Dice = 0, aucune erreur.
- **Masque prédit à côté du vrai anneau, ou vide** : Dice = 1, erreur maximale.
- Les pixels de fond n'apparaissent pas dans la formule : Dice n'est donc pas trompé par les 99 % de fond. C'est le complément idéal du terme précédent.

Il n'est calculé que sur les images qui contiennent un anneau: sur une image vide, il n'y a rien à recouvrir.

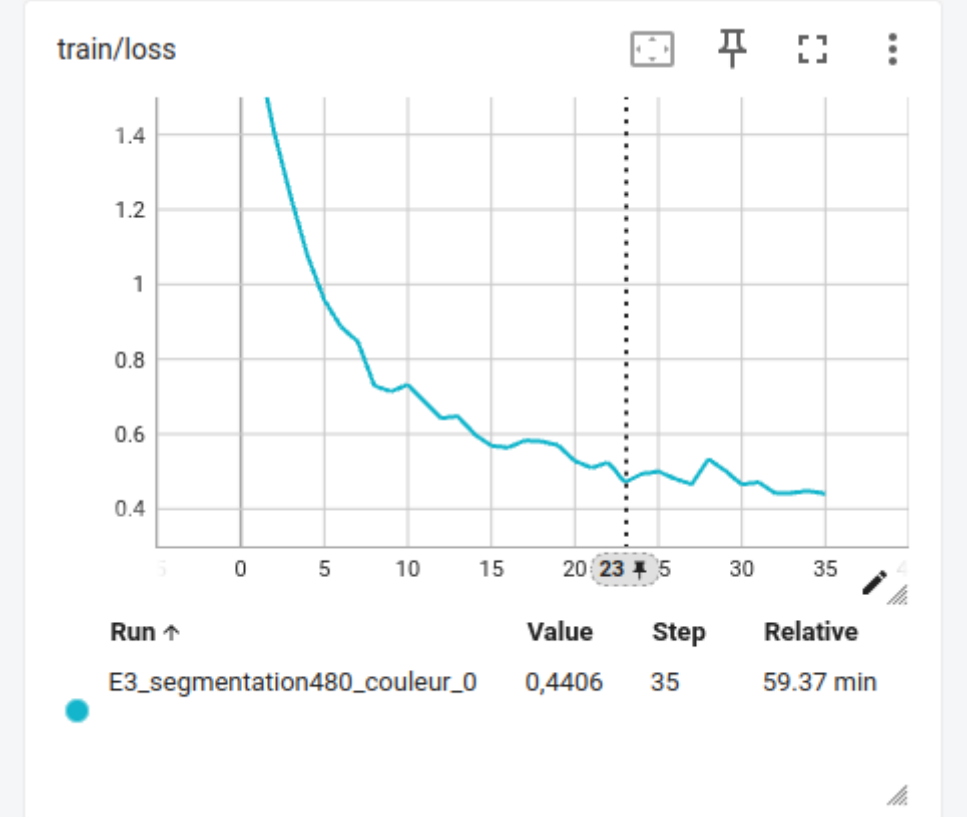
En résumé

Terme	Question posée	Calculé sur	Particularité
Présence	Y a-t-il un cerceau ?	toutes les images	—
Pixel par pixel	Chaque pixel est-il du tube ?	images avec masque connu (vides comprises)	erreurs sur le tube × 10
Dice	Le masque recouvre-t-il l'anneau ?	images avec un anneau	insensible au fond

3.5 L'entraînement

L'entraînement consiste à ajuster les 1,67 million de paramètres du réseau de façon à minimiser la perte définie au paragraphe précédent. Les 495 images d'entraînement sont présentées par lots de 16 ; après chaque lot, les poids sont mis à jour par descente de gradient. Un passage complet sur l'ensemble des images constitue une époque. À l'issue de chaque époque, le modèle est évalué sur les 120 images de validation, exclues de l'apprentissage : cette évaluation permet de suivre la capacité de généralisation du réseau et de détecter l'apparition du surapprentissage. Le modèle retenu est celui qui obtient la plus faible perte de validation.

Réglage	Valeur
Optimiseur	AdamW, pas 10^{-3} pour le décodeur et les têtes, 10^{-4} pour l'encodeur (× 0,1), pénalité des poids 10^{-4}
Calendrier du pas (variation du coefficient d'apprentissage)	Cosinus sur 50 époques : pas = pas initial × (1 + cos(π·époque/50)) / 2
Lot	16 images
Nombre d'époques	50 au maximum ; arrêt anticipé après 12 époques sans amélioration
Choix du meilleur modèle	Plus petite perte de validation



Nous voyons que la meilleure époque correspond à l'époque 23 (perte de validation de 0.6827) car à partir de ce moment l'erreur ne baisse plus mais stagne (0.7 à 0.73) voire remonte, alors que la perte d'entraînement continue de baisser : le réseau commence à sur-apprendre. L'arrêt anticipé s'est déclenché après l'époque 35 (12 époques sans amélioration), et c'est le modèle de l'époque 23 qui a été conservé. Sur le processeur d'un ordinateur portable, l'entraînement a duré environ une heure (102 s par époque).

3.6 Comparaison des modèles

Bien entendu pour en arriver à un résultat convenable, nous avons essayé plusieurs modèles, avec des augmentations voire des structures différentes (E0, E1, E2, E3).

Pour guider le drone, le réseau doit répondre à deux questions : y a-t-il un cerceau dans l'image ? et où se trouve-t-il exactement ? La première relève de la classification ; la seconde, la localisation, peut être traitée de deux manières différentes. Les modèles E0 et E1 utilisent la régression, les modèles E2 et E3 la segmentation.

3.7 La régression (E0, E1)

Le réseau de régression produit directement les paramètres de l'ellipse : une probabilité de présence, le centre (cx, cy), les deux demi-axes (A, B) et l'angle d'inclinaison.

```
image → encodeur → couche dense → 6 nombres (présence, cx, cy, A, B, angle)
```

L'apprentissage consiste à rapprocher ces six nombres des valeurs annotées. Cette approche est simple et ne demande aucun traitement supplémentaire après le réseau.

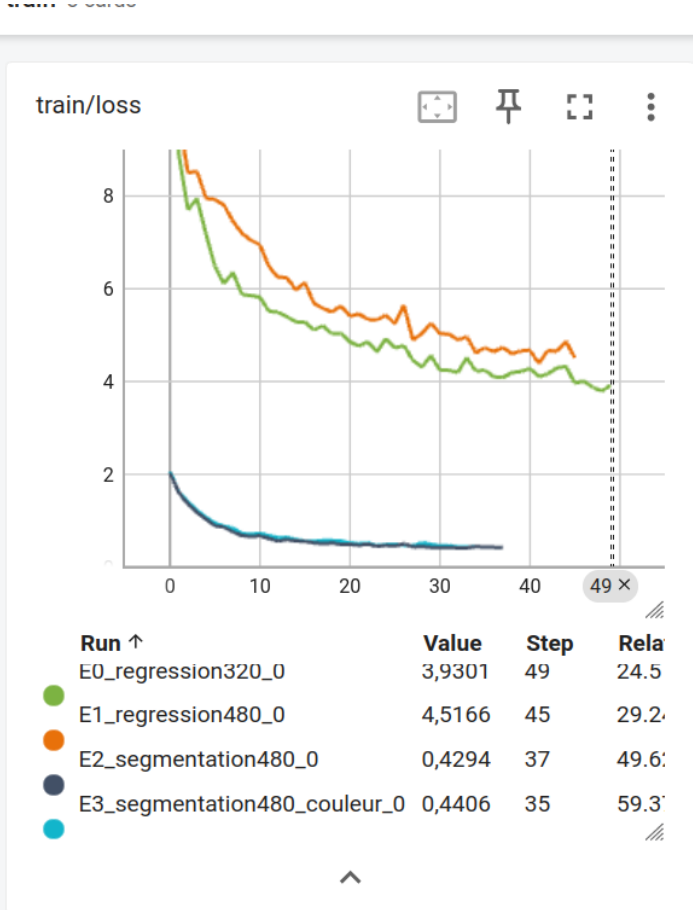
Sa limite tient à la dernière couche, dite dense : elle associe un poids différent à chaque position de l'image. Pour localiser correctement un cerceau situé dans une zone donnée, le réseau doit en avoir rencontré suffisamment à cet endroit pendant l'entraînement. Avec un jeu de données réduit, il généralise mal et, en cas d'incertitude, tend à répondre une position « moyenne », souvent proche du centre de l'image.

3.8 La segmentation (E2, E3)

Le réseau de segmentation ne prédit pas l'ellipse : il indique, pour chaque pixel, la probabilité qu'il appartienne au tube du cerceau. Il produit ainsi un masque, à partir duquel l'ellipse est ensuite calculée par un ajustement géométrique, sans apprentissage.

```
image → encodeur → décodeur → masque (probabilité « tube » pour chaque pixel)
                                  └→ ajustement d'ellipse → centre, demi-axes, angle
```

L'apprentissage compare le masque prédit au masque cible, dessiné automatiquement à partir de l'ellipse annotée. Les mêmes filtres de convolution étant appliqués sur toute l'image, un motif de tube appris à un endroit est reconnu partout ailleurs. Chaque image fournit en outre plusieurs dizaines de milliers de pixels à apprendre, contre six valeurs pour la régression : le réseau tire beaucoup plus d'information d'un même nombre d'images.



3.9 Résultats sur la validation

Modèle	Présence (exactitude)	Erreur médiane du centre	Erreur médiane de taille
E0 – régression 320×240	89,2 %	68 px	16,6 %
E1 – régression 480×360	90,8 %	78 px	28,1 %
E2 – segmentation 480×360	95,0 %	9,2 px	3,9 %
E3 – segmentation 480×360, couleur	95,8 %	9,7 px	3,0 %

La détection de présence est comparable entre les deux approches. L'écart porte sur la localisation : la segmentation divise l'erreur sur le centre par sept environ et réduit fortement l'erreur sur la taille, dont dépend directement l'estimation de la distance. C'est cette précision, indispensable à la pose 3D et au guidage du drone, qui a motivé le passage à la segmentation à partir du modèle E2. Enfin, le réseau E2 ne suffisait pas à lui seul : toutes ses images d'entraînement montraient le même cerceau violet, et il pouvait donc s'appuyer sur sa couleur. Or, lors de nos essais, nous avons dû utiliser un second cerceau, vert, qu'E2 avait du mal à reconnaître. Plutôt que de photographier et d'annoter ce nouveau cerceau, ce qui n'aurait appris au réseau qu'une couleur de plus, nous avons modifié les augmentations de nos images déjà annotées : rotation de la teinte, permutation des canaux et passage en niveaux de gris. Résultat : E3, un réseau beaucoup moins sensible à la couleur, qui s'appuie davantage sur la forme du cerceau et produit nettement moins de fausses alarmes quand les couleurs de la scène changent (voir la partie Robustesse à la couleur).

3.10 Métriques et résultats

Toutes les mesures portent sur les 120 images de validation (90 avec cerceau, 30 sans), jamais vues pendant l'entraînement.

Définitions des métriques

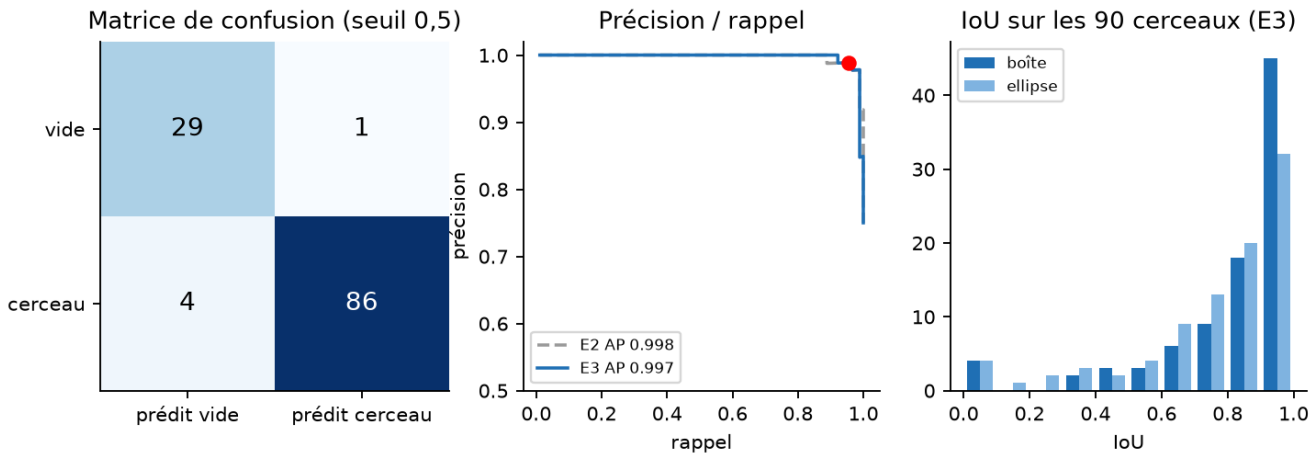
Présence (classification). Le réseau donne une probabilité p ; on décide « cerceau » si p ≥ 0,5. Chaque image tombe dans une des quatre cases de la matrice de confusion :

Case	Signification	Conséquence pour le drone
VP (vrai positif)	cerceau présent, détecté	—
FN (faux négatif)	cerceau présent, raté	le drone continue à chercher : perte de temps
FP (faux positif)	rien, mais cerceau annoncé	le drone vise un objet qui n'existe pas : dangereux
VN (vrai négatif)	rien, et rien annoncé	—

- **Exactitude** = (VP + VN) / total : part des images bien classées.
- **Précision** = VP / (VP + FP) : quand le modèle annonce un cerceau, peut-on le croire ? Métrique de sécurité.
- **Rappel** = VP / (VP + FN) : parmi les vrais cerceaux, combien sont trouvés ?
- **Spécificité** = VN / (VN + FP) : parmi les images vides, combien sont reconnues vides ?
- **F1** = 2 · précision · rappel / (précision + rappel).
- **AP** (average precision) : aire sous la courbe précision / rappel obtenue en faisant varier le seuil ; juge le modèle indépendamment du seuil (1 = parfait).

Localisation. Le recouvrement entre forme prédite et forme de référence se mesure par l'IoU (Intersection over Union) = aire commune / aire totale couverte (1 = identiques). On le calcule sur le masque (pixels « tube »), la boîte englobante et l'ellipse pleine. S'y ajoutent les erreurs utiles au vol : erreur du centre (px), erreur relative du grand axe (elle se transmet telle quelle à la distance), erreur du rapport B/A (orientation du cerceau) et erreur d'inclinaison de l'ellipse (degrés, ellipses aplaties B/A < 0,85).

Présence



Gauche : matrice de confusion d'E3 au seuil 0,5. Milieu : courbe précision / rappel de la présence (point rouge : seuil 0,5).

Droite : IoU boîte et ellipse sur les 90 images avec cerceau (une image sans prédiction compte 0).

Métrique	E3	Lecture (E3)
Matrice de confusion	VP 86 · FN 4 · FP 1 · VN 29	5 erreurs sur 120
Exactitude	95,8 %	115 images sur 120
Précision	98,9 %	1 seule fausse alarme
Rappel	95,6 %	4 cerceaux ratés
Spécificité	96,7 %	29 images vides sur 30
F1	0,972	
AP (aire sous P/R)	0,997	les probabilités classent presque parfaitement les images

Localisation 2D : la précision de l'ellipse

Ce tableau compare l'ellipse prédite par E3 à l'ellipse annotée, sur les images de validation où le cerceau est détecté. La médiane donne l'erreur typique ; la moyenne est tirée vers le haut par les quelques grosses erreurs.

Mesure	E3
IoU du masque	0,45
Centre 2D, médiane	9,7 px
Centre 2D, moyenne	22,2 px
Centres à plus de 50 px	8 ellipses sur 86
Grand axe (≈ distance), médiane	3,0 %
Grand axe, moyenne	14,4 %
Rapport B/A, erreur moyenne	0,074
Inclinaison (ellipses aplaties), médiane	20,1°

L'IoU du masque paraît faible, car l'anneau ne fait que quelques pixels d'épaisseur : un décalage d'un pixel suffit à la réduire fortement, sans affecter la précision de l'ellipse. L'inclinaison n'est mesurée que sur les 26 ellipses nettement aplaties (B/A < 0,85), les seules pour lesquelles cet angle a un sens.

- **Centre :** l'erreur typique est inférieure à 10 px sur une image de 960 px de large, soit environ 1 % de la largeur.
- **Taille :** le grand axe est estimé à 3 % près en médiane, ce qui donne une distance précise à environ 3 % ; l'écart avec la moyenne (14 %) vient de quelques cerceaux lointains mal mesurés.
- **Forme et orientation :** l'aplatissement (B/A) et l'inclinaison sont bien restitués, ce qui permet d'estimer sous quel angle le cerceau est vu.

Pourquoi une ellipse plutôt qu'une boîte englobante

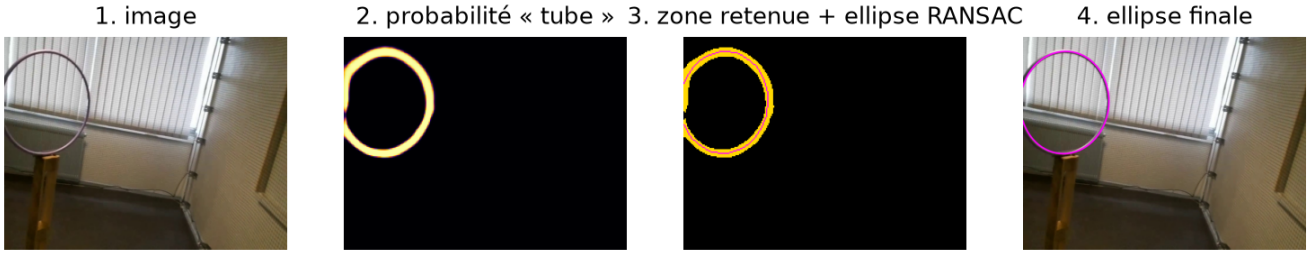
Les détecteurs d'objets classiques localisent un objet par une boîte englobante : le plus petit rectangle aligné sur l'image qui le contient. Une boîte ne suffit pas ici : elle donne la position et la taille du cerceau, mais pas son orientation. Un cerceau vu de face et le même cerceau vu de biais peuvent avoir presque la même boîte. Or la position 3D a besoin de cette orientation, qui est

contenue dans l'ellipse (rapport B/A et inclinaison).

E3 prédit donc une ellipse, dont on peut toujours déduire une boîte si besoin. Pour se comparer aux détecteurs classiques, on mesure cette boîte avec leur critère habituel : une détection est jugée correcte si sa boîte recouvre la boîte de référence à au moins 50 % (IoU ≥ 0,5). On calcule alors la précision moyenne du modèle sur l'ensemble des seuils de confiance, c'est-à-dire l'aire sous sa courbe précision / rappel : elle vaut 1 si tous les cerceaux sont trouvés et bien placés, sans fausse alarme.

E3 obtient un IoU médian de 0,90 et une précision moyenne de 0,89 : la grande majorité des cerceaux est à la fois détectée et correctement placée.

3.11 Du masque aux coordonnées 2D



La chaîne d'E3 sur une image de la cage. 2 : probabilité « tube » prédite. 3 : zone retenue (jaune) et ellipse RANSAC (magenta). 4 : résultat final.

Ajuster l'ellipse sur le masque

Le masque prédit n'est pas une ellipse parfaite : l'anneau peut être incomplet (cerceau coupé par le bord de l'image ou caché), et quelques pixels isolés peuvent être coloriés à tort. Il faut donc retrouver l'ellipse qui explique le mieux les pixels du tube, sans se laisser tromper par les pixels parasites.

1. **Seuil :** on garde les pixels dont la probabilité « tube » est au moins 0,5.
2. **Plus grande zone connexe :** seule la plus grande tache est conservée ; les petites taches isolées sont considérées comme du bruit.
3. **Deux ellipses candidates** sont calculées sur cette zone (voir ci-dessous).
4. **Choix :** on retient la candidate dont le contour passe sur les pixels les plus probables (probabilité moyenne sur 128 points de son contour).
5. **Changement de repère :** l'ellipse est ramenée du masque (240×180) à l'image de la caméra (960×720), soit un facteur 4.

Candidate A : la méthode RANSAC

Un ajustement classique (moindres carrés) utilise tous les pixels à la fois : quelques pixels parasites suffisent alors à déformer l'ellipse. La méthode RANSAC (*RANdom SAmple Consensus*, « consensus sur échantillons aléatoires ») évite ce problème en cherchant l'ellipse sur laquelle le plus grand nombre de pixels est d'accord :

1. on tire 5 pixels au hasard dans la zone ; 5 points suffisent à définir une ellipse ;
2. on calcule l'ellipse qui passe par ces 5 points ;

3. on compte les pixels qui la soutiennent, c'est-à-dire ceux situés à moins de 4 % de son contour ;
4. on recommence 150 fois et on garde l'ellipse qui a réuni le plus de pixels ;
5. on la réajuste sur ses seuls pixels de soutien, en ignorant les autres.

Les pixels parasites ne sont jamais d'accord entre eux : une ellipse tirée à partir d'eux ne réunit que très peu de soutiens et est écartée. La méthode fonctionne bien quand le masque est un anneau ou un simple arc, par exemple un cerceau partiellement visible.

Candidate B : le contour extérieur

Pour un cerceau lointain, l'anneau devient si fin qu'il se transforme en une petite tache pleine, sur laquelle RANSAC n'a plus de sens. On ajuste alors une ellipse sur le bord extérieur de la tache, réduite d'une demi-épaisseur de tube pour retrouver le milieu de l'anneau.

Lors des essais en vol, quelques fausses ellipses sont apparues : lorsque la plus grande zone du masque correspond à une main, une manche ou une perche, l'ellipse est ajustée sur cet objet, devient très allongée et donne une distance fausse. Un filtre a donc été ajouté après l'ajustement. Une ellipse est rejetée, et l'image traitée comme sans cerceau, si moins de 35 % de son contour passe sur l'anneau prédit (ellipse ajustée sur un fragment), ou si elle est très plate (rapport B/A inférieur à 0,35) alors que le cerceau est estimé à plus de 2,5 m. Cette condition de distance évite d'écarter de vrais cerceaux vus de biais de près.

Le centre 2D du cerceau n'est pas le centre de l'ellipse

Par la perspective, la moitié la plus proche du cerceau paraît plus grande : le centre de l'ellipse n'est pas l'image du centre du cerceau. Le centre 2D renvoyé (u, v) est donc la projection du centre 3D, distorsion comprise. Dans la validation de la cage, l'écart vaut 5,6 px en médiane et jusqu'à 48 px, de près.

3.12 Des coordonnées 2D à la 3D

La calibration de la caméra

Pour passer d'une position en pixels à une position en mètres, il faut connaître la manière dont la caméra du drone projette le monde sur l'image. C'est le rôle de la calibration.

Le modèle de la caméra. La caméra est modélisée comme un sténopé : un point situé à X mètres à droite, Y mètres en dessous et Z mètres devant l'objectif apparaît dans l'image au pixel

```
u = fx · X / Z + cx
v = fy · Y / Z + cy
```

- fx et fy sont les focales, exprimées en pixels : elles indiquent de combien de pixels se déplace un objet quand on s'écarte de l'axe ;
- (cx, cy) est le centre optique : le pixel qui correspond à l'axe de la caméra.

Par exemple, un point situé à 1 m devant la caméra et 10 cm sur sa droite apparaît à u = 927 × 0,1 / 1 + 476 ≈ 569 px, soit environ 93 pixels à droite du centre optique.

Les coefficients de distorsion :

Un objectif réel ne se comporte pas exactement comme un sténopé : les lignes droites apparaissent légèrement courbées, surtout vers les bords de l'image. Cette déformation est décrite par cinq coefficients, de deux natures.

- **Distorsion radiale (k1, k2, k3).** Elle dépend uniquement de la distance r au centre de l'image : les points sont rapprochés du centre ou éloignés de lui, d'autant plus qu'ils en sont loin. Selon le signe des coefficients, un carré prend une forme de tonneau (bords bombés vers l'extérieur) ou de coussinet (bords creusés vers l'intérieur).
- **Distorsion tangentielle (p1, p2).** Elle vient d'un léger défaut d'alignement entre l'objectif et le capteur, qui ne sont pas parfaitement parallèles. L'image est alors légèrement inclinée d'un côté.

En coordonnées normalisées ($x = (u - cx) / fx$, $y = (v - cy) / fy$, et $r^2 = x^2 + y^2$), un point (x, y) est déplacé vers :

```
x' = x · (1 + k1·r² + k2·r⁴ + k3·r⁶) + 2·p1·x·y + p2·(r² + 2·x²)
y' = y · (1 + k1·r² + k2·r⁴ + k3·r⁶) + p1·(r² + 2·y²) + 2·p2·x·y
         └──────── distorsion radiale ────────┘   └──── distorsion tangentielle ────┘
```

Pour la caméra du Tello, ces coefficients sont très petits (k1 = −0,015 ; k2 = 0,035 ; k3 = 0,034 ; p1 = 0,002 ; p2 = 0,002). Dans un coin de l'image, là où la distorsion est la plus forte, le facteur radial vaut environ 1,002, ce qui décale le point de 1 à 2 pixels seulement. Au centre de l'image, la distorsion est nulle. Cette correction reste utile pour les cerceaux proches, qui occupent une grande partie de l'image et dont le contour atteint les bords.

La procédure: Un damier imprimé, de 9×6 coins intérieurs et de cases de 27 mm, a été photographié 39 fois par la caméra du drone, sous des angles et à des distances variés . Sur chaque photo, OpenCV détecte les 54 coins au dixième de pixel près. Comme leur position réelle sur le damier est connue exactement, on peut chercher les paramètres de la caméra qui expliquent le mieux les coins observés.

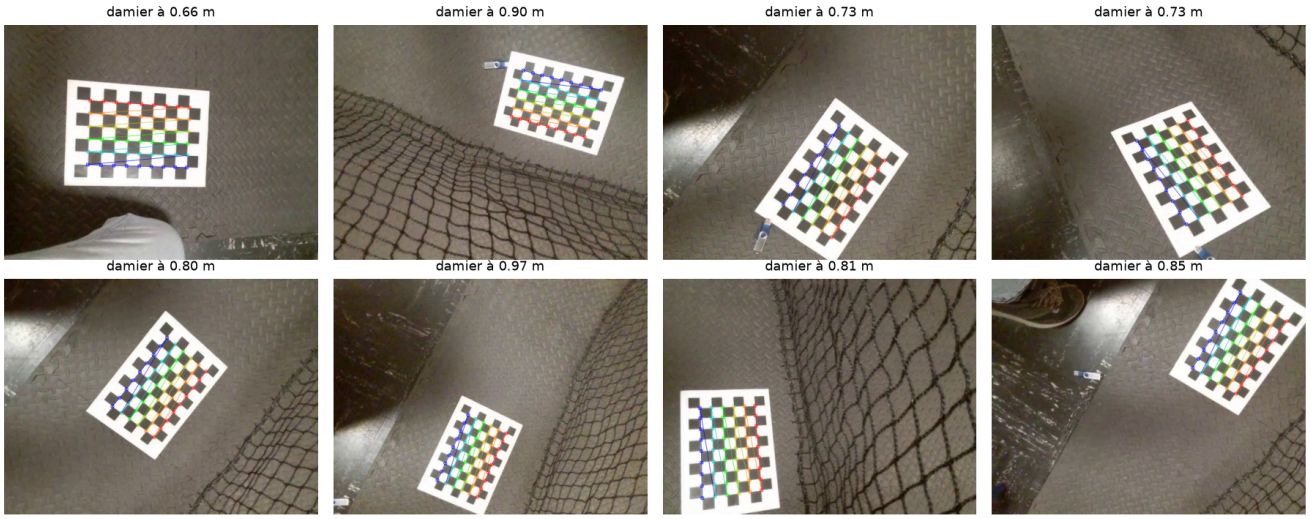


Figure – Exemple de photo de calibration, avec les coins du damier détectés.

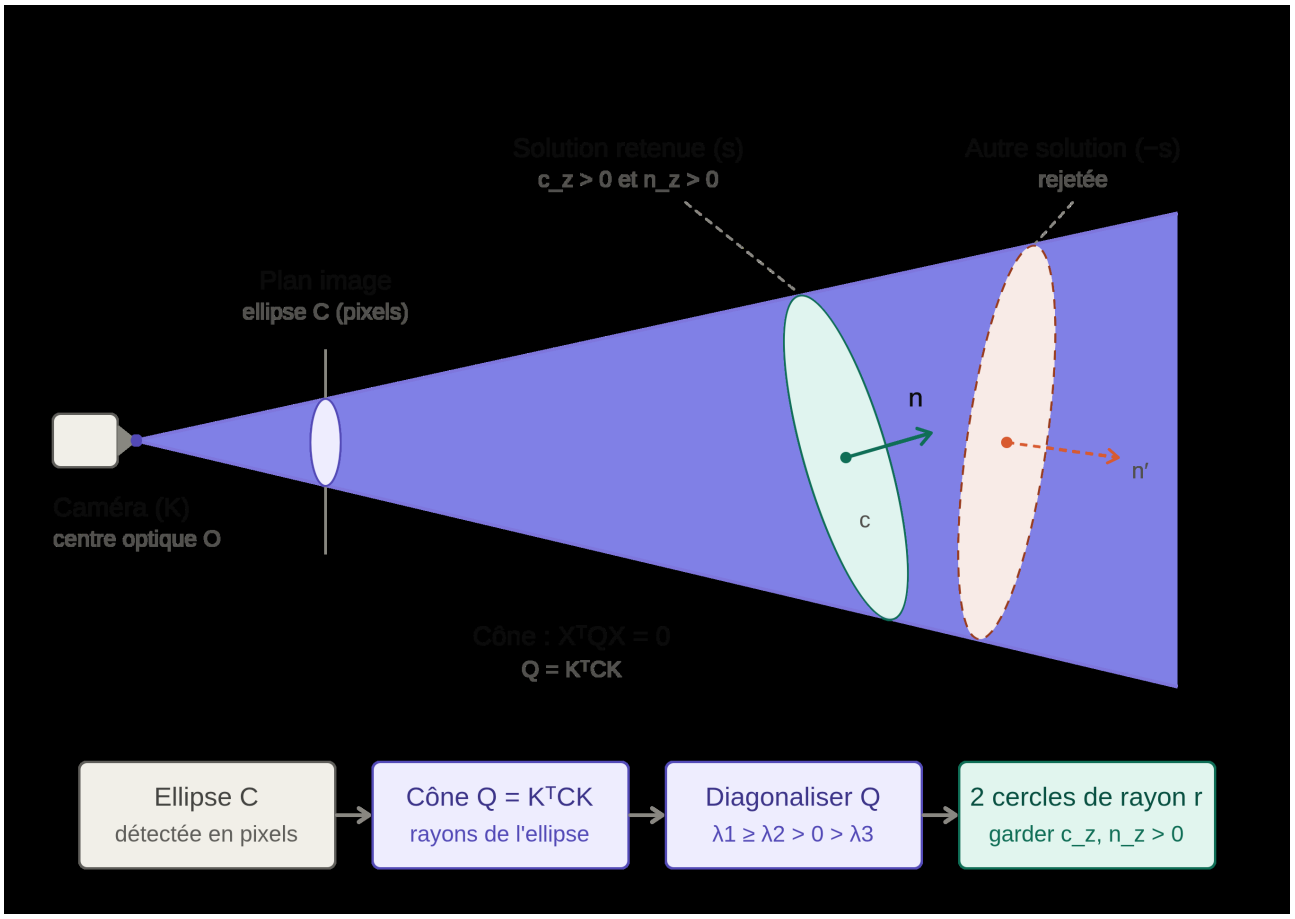
Résultats.

Grandeur	Valeur	Sens
Focales fx, fy	927,1 et 925,8 px	presque égales : les pixels sont carrés
Centre optique cx, cy	475,8 ; 386,3 px	proche du centre de l'image (480 ; 360)
Distorsion k1, k2, p1, p2, k3	−0,015 ; 0,035 ; 0,002 ; 0,002 ; 0,034	très faible
Erreur de reprojection	0,23 px	en moyenne, le modèle replace chaque coin à un quart de pixel de sa position observée
Champ de vision	54,7° × 42,5°	déduit des focales : 2 · arctan(480 / 927) = 54,7°

L'erreur de reprojection de 0,23 px indique une calibration de très bonne qualité. La distorsion étant faible, elle est corrigée par calcul sur le contour de l'ellipse, plutôt que sur toute l'image.

Du cercle vu en ellipse au cercle 3D

Le cerceau est un cercle de rayon connu, **r** = **0,34** m. Vu par la caméra sous un angle quelconque, il apparaît comme une ellipse. On cherche à retrouver, à partir de cette seule ellipse, la position du cerceau dans l'espace : son centre **c** (en mètres, dans le repère de la caméra) et sa normale **n**, c'est-à-dire la direction dans laquelle il faut le traverser.



1. Le cône de vision. Chaque pixel de l'image correspond à un rayon lumineux qui part de la caméra et file dans l'espace. Les rayons qui passent par tous les points de l'ellipse forment un cône, dont le sommet est la caméra. Le vrai cerceau est forcément posé quelque part sur ce cône, puisque c'est lui qui a produit l'ellipse.

Mathématiquement, l'ellipse s'écrit :

$$\mathbf{p}^\top C\,\mathbf{p} = 0 \quad ; \quad \mathbf{p} = (u, v, 1)^\top$$

Ici, **p** est un pixel et **C** une matrice 3×3 qui résume la forme de l'ellipse. La calibration fournit la matrice **K** qui relie les pixels aux directions dans l'espace. Le cône s'obtient alors simplement :

$$\mathbf{X}^\top Q\,\mathbf{X} = 0, \qquad Q = K^\top C\,K,$$

où **X** est un point de l'espace, en mètres. Cette équation dit seulement : « **X** se projette sur l'ellipse ».

2. Redresser le cône. Le cône est en général penché par rapport à la caméra. Pour l'étudier plus facilement, on se place dans un repère aligné sur ses axes, en diagonalisant **Q** : on obtient trois nombres :

$$\lambda_1 \geq \lambda_2 > 0 > \lambda_3$$

Trois nombres qui décrivent son ouverture dans chaque direction, et une matrice de rotation **V** qui permet de revenir au repère de la caméra. Dans ce repère, l'équation du cône devient très simple :

$$\lambda_1 x^2 + \lambda_2 y^2 + \lambda_3 z^2 = 0.$$

3. Où couper le cône pour obtenir un cercle ? Couper un cône par un plan donne en général une ellipse. Mais il existe exactement deux orientations de plan pour lesquelles la coupe est un cercle parfait. Ce sont les deux orientations possibles du cerceau. Leurs normales s'écrivent, dans le repère du cône :

$$\mathbf{n}' = \begin{pmatrix} s\,a \\ 0 \\ -b \end{pmatrix}, \qquad a = \sqrt{\frac{\lambda_1 - \lambda_2}{\lambda_1 - \lambda_3}}, \qquad b = \sqrt{\frac{\lambda_2 - \lambda_3}{\lambda_1 - \lambda_3}}, \qquad s = \pm 1.$$

Le signe **s** distingue les deux solutions.

4. La taille fixe la distance. Plus on coupe le cône loin de la caméra, plus le cercle obtenu est grand. Comme on connaît le vrai rayon du cerceau (0,34 m), il n'existe qu'une seule distance à laquelle la coupe a exactement cette taille. On en déduit le centre, puis on revient au repère de la caméra avec **V** :

$$\mathbf{n} = V \begin{pmatrix} s\,a \\ 0 \\ -b \end{pmatrix}, \qquad \mathbf{c} = \frac{r}{\sqrt{-\lambda_1 \lambda_3}}\, V \begin{pmatrix} s\,a\,\lambda_3 \\ 0 \\ -b\,\lambda_1 \end{pmatrix}.$$

Le centre **c** est proportionnel au rayon **r** : si le diamètre déclaré dans `configs/hoop.yaml` (0,68 m) était faux de 5 %, toutes les distances le seraient aussi de 5 %. La normale **n**, elle, ne dépend pas de la taille du cerceau.

5. Choisir entre les deux solutions. On ne garde que les solutions physiquement possibles : cerceau devant la caméra (**c_z > 0**) et normale orientée dans le sens de la traversée (**n_z > 0**). Mais il reste presque toujours deux cercles possibles, de même centre et d'orientations différentes, comme une pièce de monnaie inclinée vers la gauche ou vers la droite, qui donne la même ellipse vue de face. On choisit alors la solution la plus proche de celle de l'image précédente, ou, à défaut, celle qui correspond à un cerceau vertical.

Validation. Testée sur 500 cercles simulés avec la vraie calibration, cette méthode retrouve le centre à 0,4 mm près et la normale à 0,13° près : le calcul géométrique est exact, et toute l'erreur finale vient de l'ellipse prédite par le réseau. Sur les images de validation de la cage (cerceau entre 0,9 et 3,1 m), la distance est retrouvée à 1,7 % près en médiane, soit environ 4 cm, et la normale à

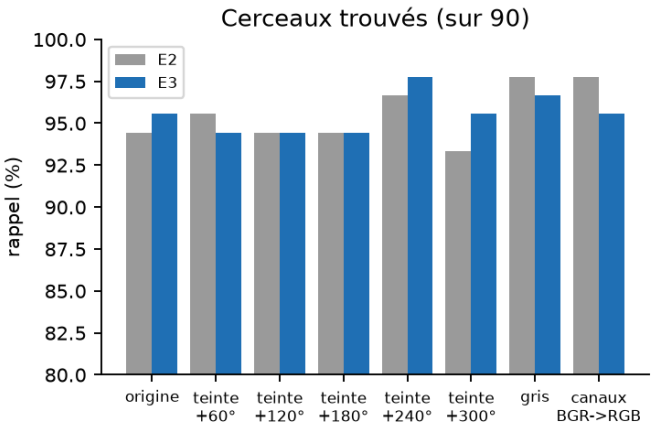
environ 7°.

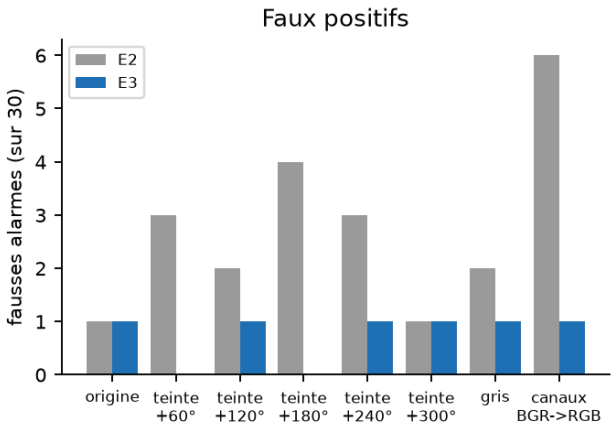
Vers le repère du drone. La caméra du Tello est inclinée de 10,3° vers le bas : la position du cerceau est donc tournée de cet angle pour l'exprimer dans le repère du drone (avant, droite, haut). C'est de ce repère que sont tirées la distance et la différence de hauteur `delta_z` envoyées au PC de contrôle ; l'écart horizontal `delta_x` est, lui, transmis en pixels.

Temps de calcul. Sur le processeur du PC vision, une image demande environ 66 ms : 54 ms pour le réseau, 11 ms pour l'ajustement de l'ellipse et 0,5 ms pour la position 3D, soit environ 15 images par seconde, suffisant pour suivre le vol du drone.

3.13 Robustesse à la couleur

Les images d'entraînement ne contiennent que le cerceau violet de la cage et des cerceaux gris. Pour mesurer l'effet des augmentations de couleur sans images d'un autre cerceau, nous avons recoloré la validation : la teinte de chaque image est décalée de 60 en 60° (le cerceau violet devient rouge, jaune, vert, cyan ou bleu), elle est convertie en niveaux de gris, ou ses canaux rouge et bleu sont inversés. On mesure alors, pour E2 et E3, le rappel sur les 90 cerceaux et les fausses alarmes sur les 30 images vides.





	E2	E3
Rappel sur les images recolorées	93 à 98 %	94 à 98 %
Fausses alarmes (7 variantes × 30 images vides)	21 / 210	5 / 210

Les deux modèles retrouvent aussi bien les cerceaux recolorés : la forme suffisait déjà à E2 pour détecter le cerceau. La différence porte sur les fausses alarmes : dès que les couleurs changent, E2 en produit jusqu'à 6 sur 30, alors qu'E3 reste à 0 ou 1. E2 avait appris à associer le cerceau aux couleurs habituelles de la scène ; E3, entraîné avec des couleurs aléatoires, s'appuie davantage sur la forme et reste fiable quelles que soient les couleurs.

En vol avec le cerceau vert, E3 le détecte avec une probabilité de 0,94 à 1,00 en dessous de 3,7 m. Deux limites sont apparues : de loin (au-delà de 4 m), le tube devient très fin dans l'image et la détection est fragile ; et une main, une manche ou une perche qui tient le cerceau peut produire une fausse ellipse, en partie écartée par le filtre décrit plus haut. Des images réelles du cerceau vert, de loin et tenu à la main, restent la meilleure façon d'améliorer ces cas.

4. Relier reconnaissance et commande

On rentre dans la partie un peu plus orientée informatique industrielle et automatisme, même si le réseau de neurones et la production de commandes corrigées restent au centre de notre étude.

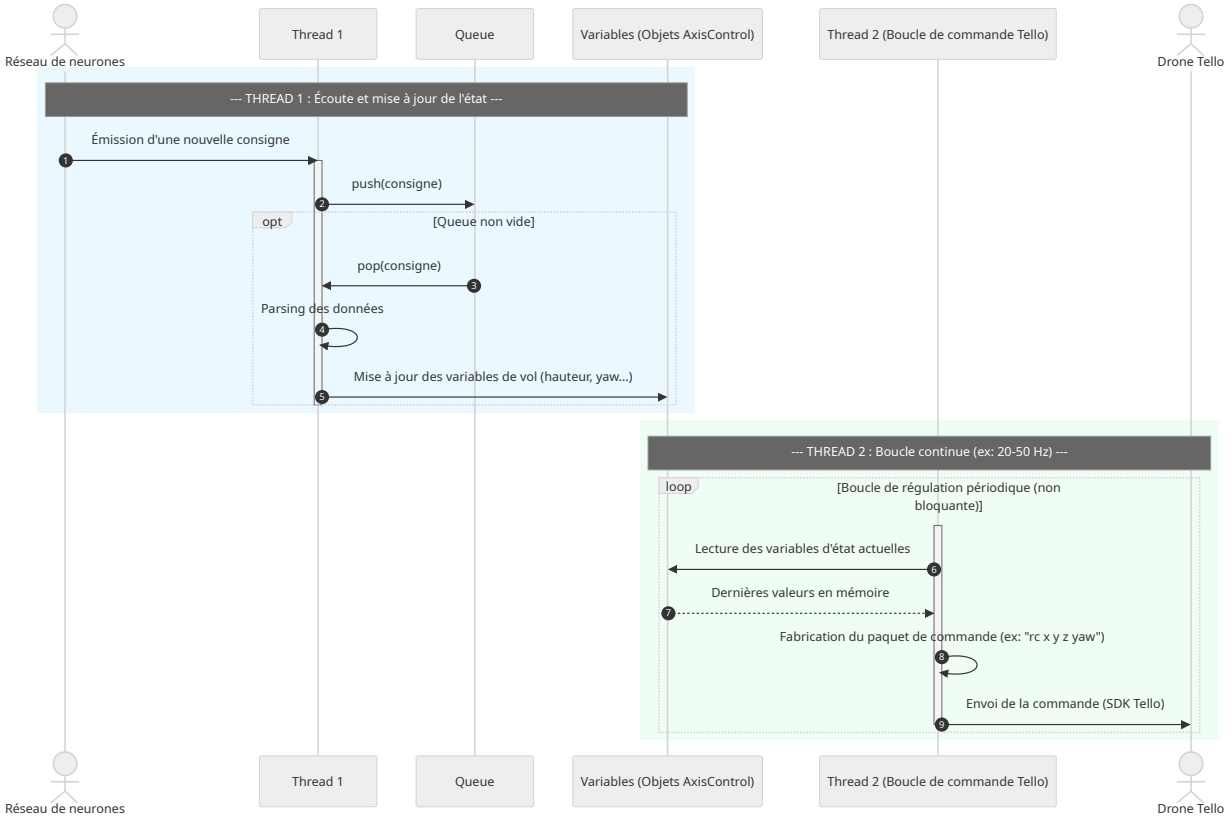
4.1 Création d'un HIL

Il a d'abord fallu restructurer le code de commande. La hauteur et le lacet étaient asservis dans des fichiers distincts, repris de nos prédécesseurs. Il fallait désormais un système capable de piloter plusieurs axes simultanément, avec des consignes qui ne sont pas connues à l'avance.

4.2 Principe du HIL

Le drone est intégré dans une boucle de type HIL (*Hardware In the Loop*), sur le modèle de fonctionnement d'une carte Arduino, où les actions périodiques sont placées dans la fonction `loop()`.

L'architecture retenue repose sur plusieurs threads :



Concrètement :

- Le thread 1 récupère les consignes et les prépare pour la boucle.
- Le thread 2, c'est le HIL, notre équivalent de `loop()` arduino.

Les commandes peuvent ainsi être produites au fil de l'eau et sont toutes prises en compte grâce à la file d'attente. Cela résout le problème du drone qui atterrissait à la fin d'une séquence de commandes préprogrammée.

Refonte du code en programmation orientée objet

Le diagramme ci-dessus fait référence à un objet `AxisControl`, issu d'un changement d'architecture. Auparavant, la hauteur et le lacet étaient asservis dans deux fichiers distincts (davantage en comptant les essais en x et en y). Le code hérité était centré sur la commande de chaque axe pris séparément, et non sur leur utilisation conjointe.

Suivant le principe *Don't Repeat Yourself* (DRY), nous avons adopté une architecture en programmation orientée objet (POO), qui évite la duplication de code entre les axes.

Une fois le code découpé en modules et validé par plusieurs dizaines de tests sur un simulateur, **un seul fichier** suffit à définir les politiques de commande de tous les axes :

```
AXIS = {
    # ...
    Y : AxisControl(
        name=Y,
        CONTROLLING = False,
        CONTROLLING_TYPE = ControllerType.P,
        K_p = 1,
        K_d = 0,
        K_i = 0,
        environment = EVT,
        max_value=40,
        min_value=-40
    ),
    YAW : AxisControl(
        name=YAW,
        CONTROLLING = True,
        CONTROLLING_TYPE = ControllerType.P,
        K_p = 2.0,
        K_d = 0,
        K_i = 0,
        environment = EVT,
        max_value=360,
        min_value=-360
    ),
    # ...
}
```

Reste à recevoir les consignes produites par la vision.

4.3 L'injection de consigne par réseau

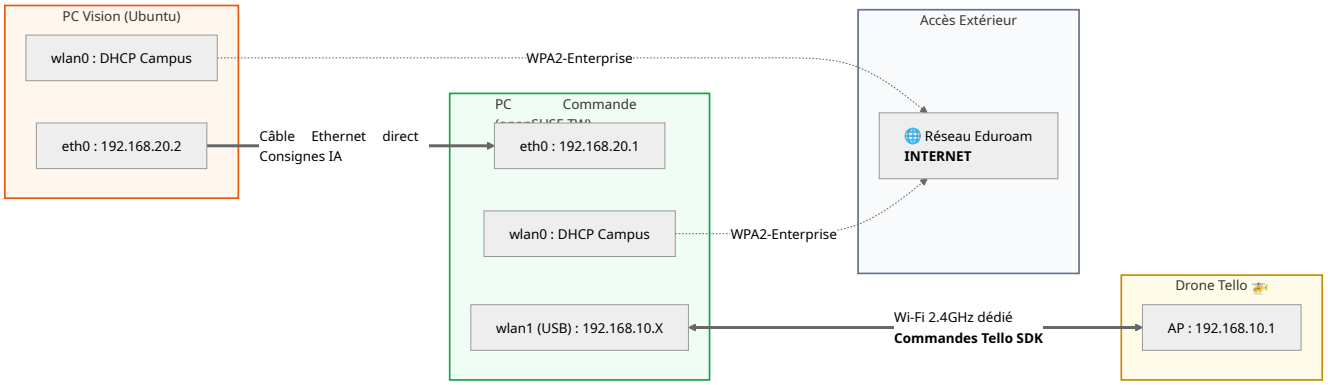
La charge de calcul (réseau de neurones, commande, visualisation) dépassait les capacités d'un seul ordinateur portable. Le système a donc été réparti sur deux ordinateurs, qu'il fallait faire communiquer.

Architecture réseau

Le matériel disponible était le suivant :

- Un ordinateur avec un réseau de neurones pytorch sur Ubuntu
- Un ordinateur avec tout le reste sur OpenSuse TW
- Deux adaptateurs RJ45 / USB-C
- Un câble Ethernet
- Une carte Wi-Fi USB

Nous avons créé un réseau dédié entre les deux PC, en plus du réseau du drone et d'Eduroam :



Cette architecture offre une faible latence entre les deux postes, l'accès à Internet et une liaison dédiée avec le drone.

Il restait à configurer le pare-feu et à vérifier l'accès aux services avec nmap.

MQTT pour la transmission de données

Les données sont transmises avec le protocole MQTT, simple, léger et adapté aux échanges temps réel.

Le broker Mosquitto est installé sur le PC de commande :

```
sudo zypper install mosquitto mosquitto-clients
```

```
sudo systemctl enable --now mosquitto
```

Le client graphique MQTT Explorer permet de visualiser les messages échangés.

Côté Python, nous utilisons le paquet `paho-mqtt` :

```
pip install paho-mqtt
```

Toujours selon le principe DRY, une classe commune aux deux postes, `MqttInterface`, est disponible dans le code source du projet. C'est elle qui permet la communication entre, non seulement le PC commande et le PC vision, mais entre le PC commande et n'importe quel client utilisant cette interface (pratique pour tester sans réseau de neurones).

Transmission du flux vidéo par MQTT

Le flux vidéo du drone transite lui aussi par MQTT, bien que ce protocole ne soit pas conçu pour ce type de données.

Au départ, nous voulions ouvrir un port UDP sur le PC commande, mais il s'est avéré beaucoup plus pratique de continuer à utiliser le SDK Tello pour récupérer la transmission et la pousser sur MQTT, que de récupérer la transmission par UDP et de la stream sur un autre port UDP. Pour l'exemple, il est possible de récupérer le flux vidéo avec :

```
echo -n "command" | nc -u -w1 192.168.10.1 8889\
echo -n "streamon" | nc -u -w1 192.168.10.1 8889
ffplay -probesize 32 -flags low_delay -i udp://0.0.0.0:11111
```

En pratique, le passage par MQTT s'est révélé plus simple à mettre en œuvre, sans pénaliser notablement la latence.

4.4 Dashboard pour la visualisation en temps réel

Consignes, mesures, état du drone, détection du cerceau, transitions d'états : de nombreux flux de données circulent dans le système, et le client MQTT ne suffisait plus à les suivre.

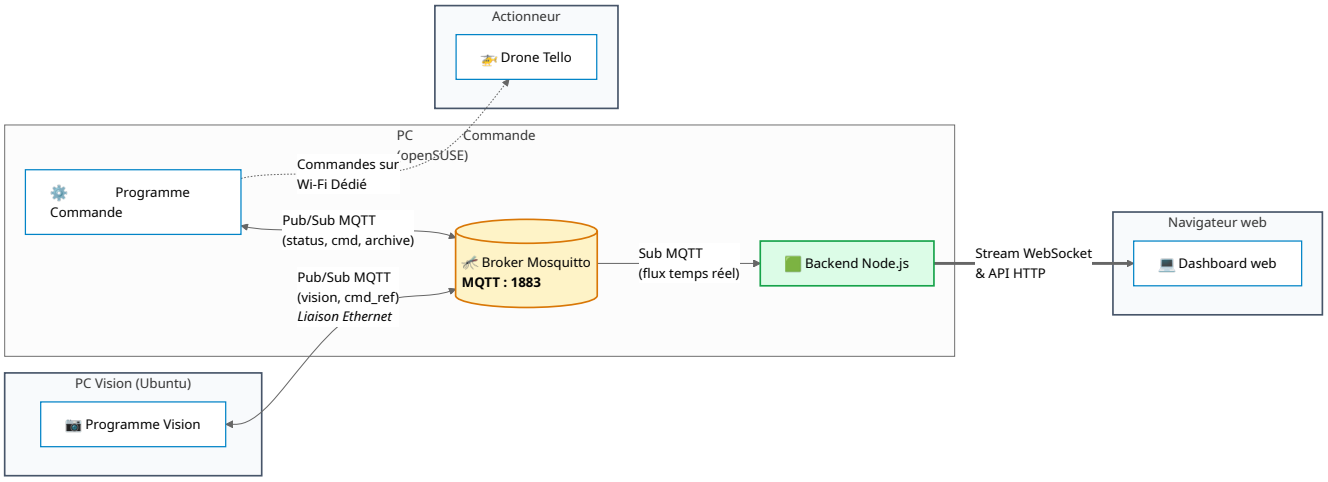
Nous avons donc développé un tableau de bord affichant ces données en temps réel.

Back-end simple & websocket

Le serveur (back-end) est volontairement minimal : un script Node.js composé d'une connexion au broker MQTT (`TelloMqttConnector.js`) et de la définition d'une websocket et de quelques routes (`server.js`).

Le terme le plus important ici est celui de websocket. En effet, comme exprimé plus haut, nous avons posé la contrainte du temps réel. Il ne sert à rien de visualiser des données vieilles de plus de 2 secondes. Donc une API REST classique montrerait vite ses limites ne serait-ce que face à la fréquence du HIL (~66 Hz). La websocket relaie directement les messages MQTT vers le front-end, dans un format exploitable par le navigateur et à haute fréquence.

Architecture des services :



La principale difficulté est de faire communiquer les services de manière robuste et de garantir la cohérence des types de données dans toute la chaîne, sous les contraintes temps réel du système cyber-physique. Le réseau étant isolé, la sécurité n'a pas été traitée.

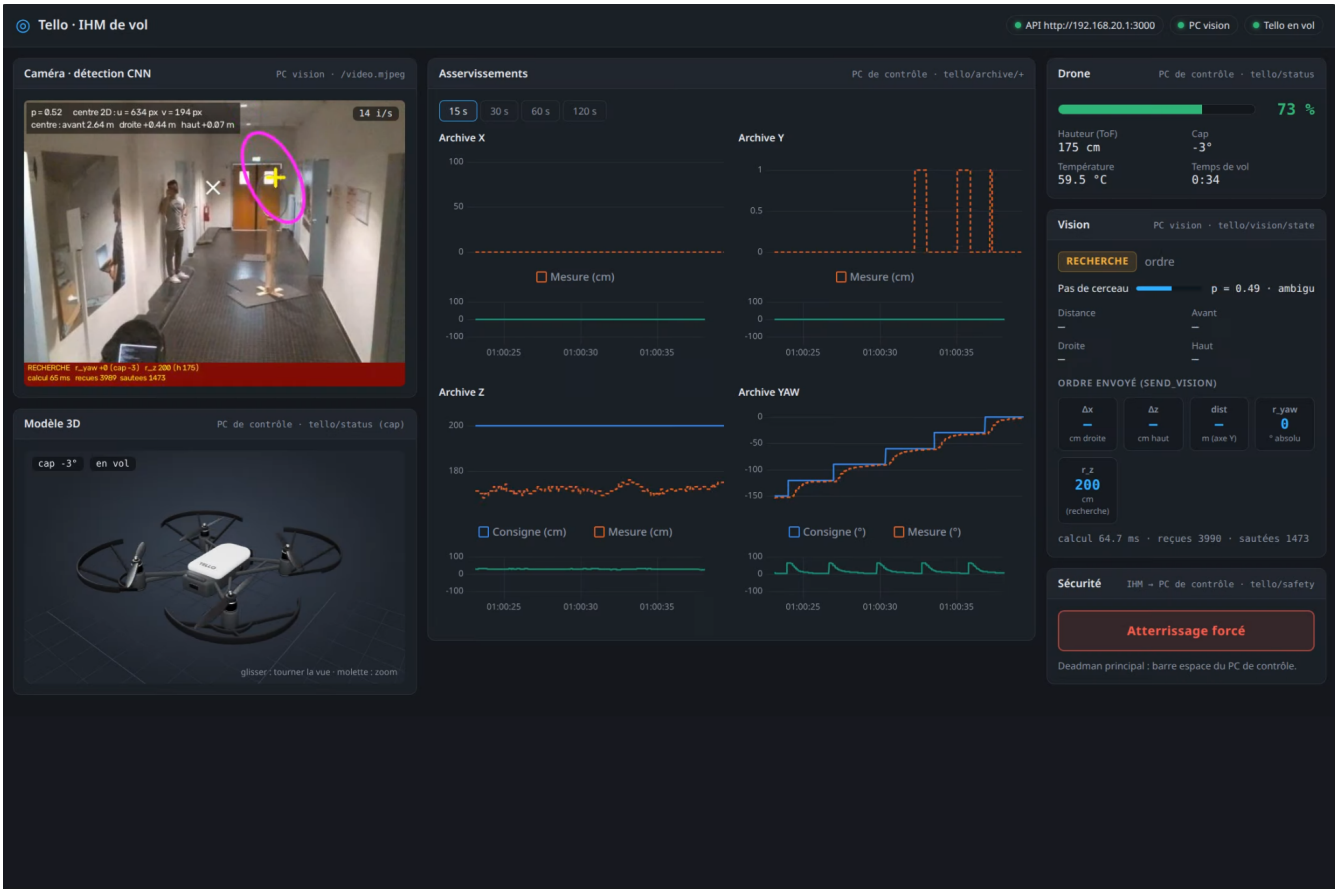
Interface web (front-end)

Le tableau de bord n'est pas le cœur du sujet, mais il est essentiel à la mise au point : pendant un vol, le réseau de neurones calcule des consignes, le poste de commande les transforme en pourcentages de vitesse et les mesures sont collectées. Chaque session est enregistrée en vidéo, puis analysée pour décider des ajustements à apporter.

Pour concentrer notre temps sur la commande et la vision, le front-end a été généré avec un assistant de programmation (Claude Code) à partir d'un cahier des charges technique. Ce choix est adapté ici car :

- les contraintes sont fixes et connues à l'avance ;
- les technologies employées sont répandues et bien documentées ;
- il s'agit d'un outil de mise au point, pas d'un produit livrable.

Spécifications retenues : React.js + Vite.js ; Sass pour le style ; JavaScript (sans TypeScript, vu la taille du projet) ; Recharts pour les graphiques ; React Three Fiber pour le modèle 3D. Notre travail a porté sur la rédaction de ces spécifications et la correction des erreurs.



Le résultat est satisfaisant ; le modèle 3D du drone reste perfectible.

4.5 FSM - Planifier les commandes

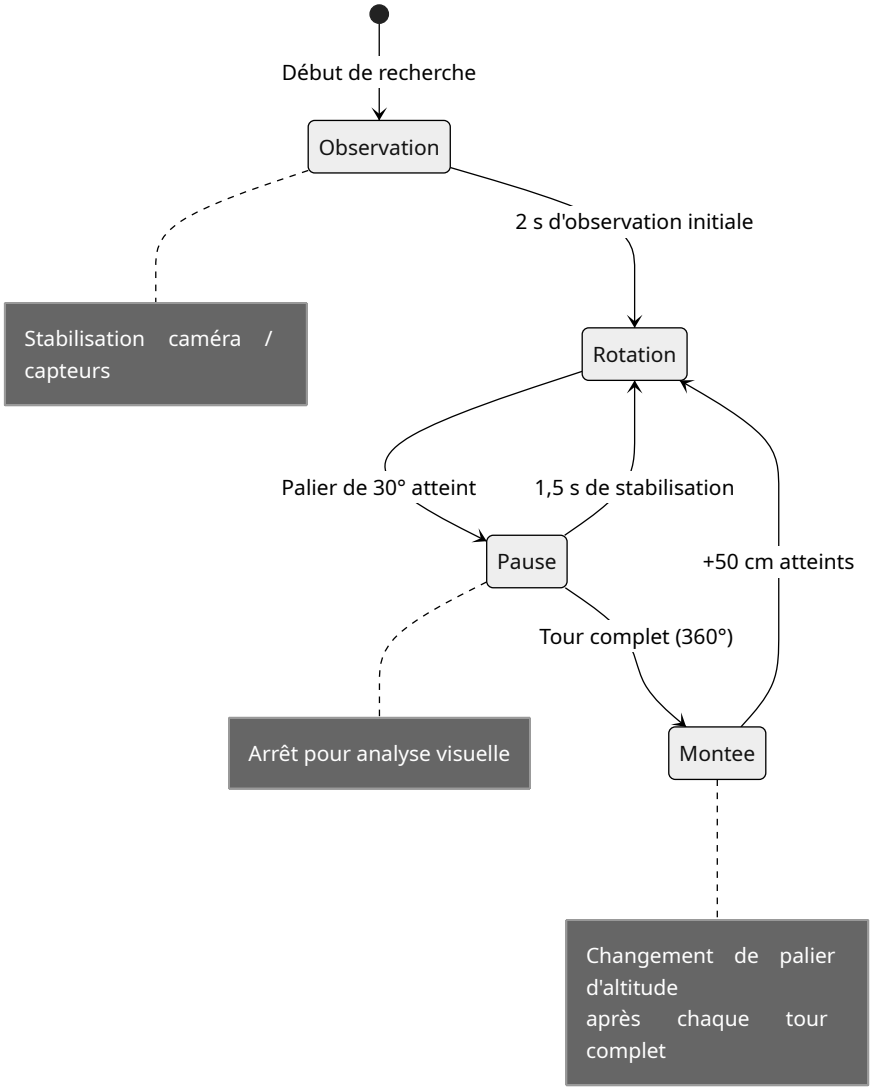
À ce stade, nous disposons de :

- L'asservissement en hauteur et en lacet du drone. On peut lui envoyer des consignes de référence sur ces deux axes et il exécutera (avec un certain temps de montée et un certain dépassement).
- Un réseau de neurones capable de détecter un cerceau et de calculer des deltas entre le centre de l'image et le centre d'un cerceau détecté (si un cerceau est détecté).
- Une communication MQTT fonctionnelle et un tableau de bord pour visualiser les détections du CNN et les réponses du drone aux consignes.

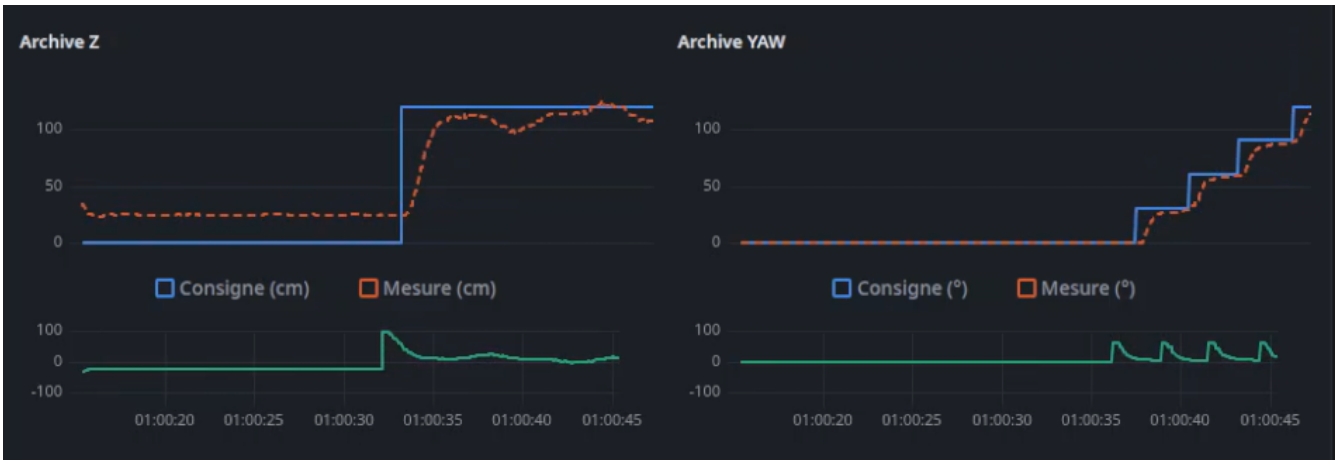
Il reste à produire des consignes à partir des données du réseau de neurones. Or le drone ne démarre jamais à la bonne hauteur ni face au cerceau : il faut séquencer plusieurs jeux de consignes, à l'aide d'une machine à états finis.

Machine à états finis pour piloter les asservissements

Un premier algorithme simple assure la recherche du cerceau :



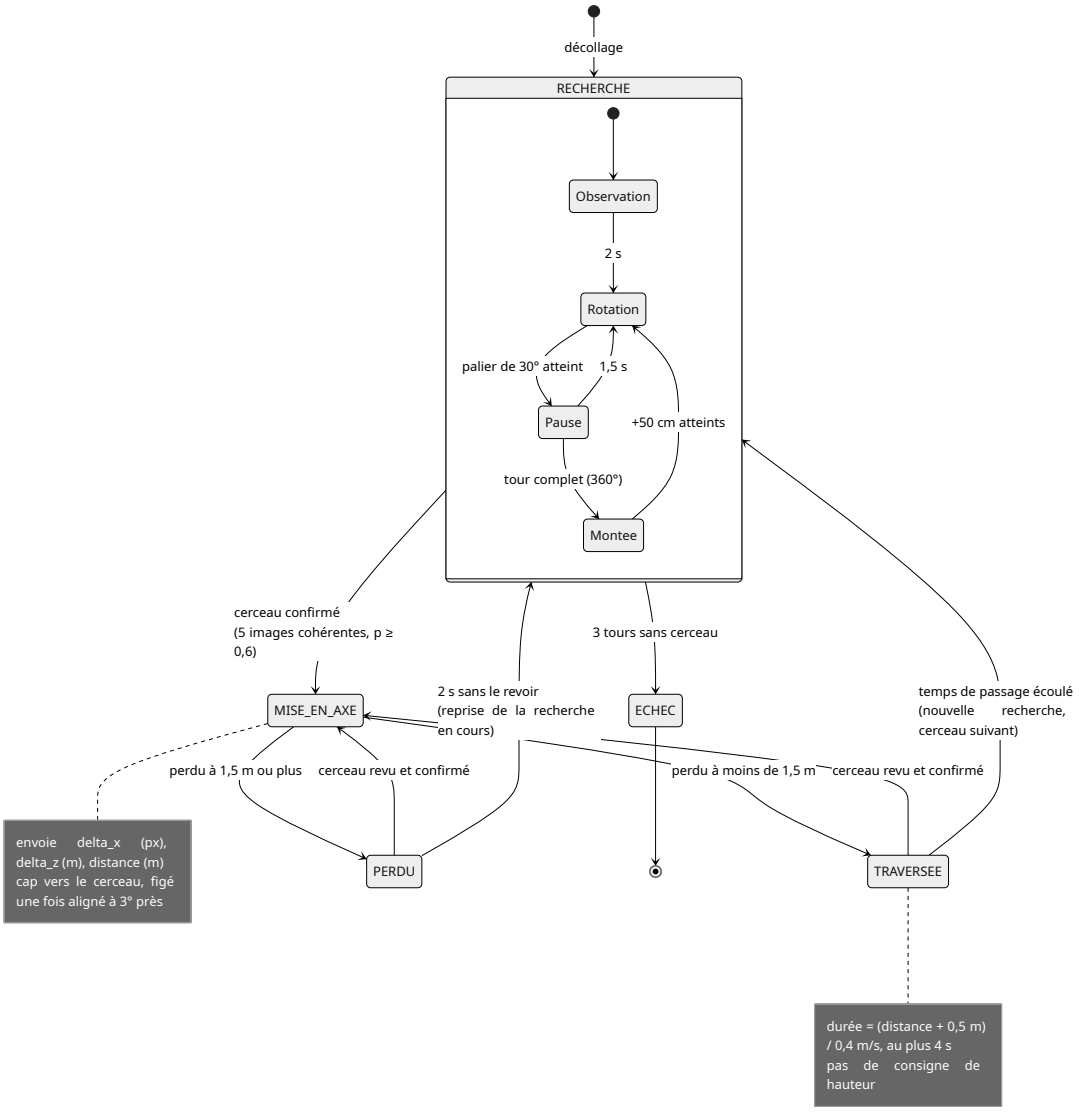
Il produit la trajectoire de recherche suivante :



Des stratégies sont ensuite définies pour chaque cas :

- Un cerceau est trouvé
- Aucun cerceau n'est trouvé après trois tours de recherche
- Un cerceau est trouvé, mais il est ensuite perdu

La machine à états complète est la suivante :



Chaque état est traduit en consignes d'axes avec l'instruction `match` - `case` de Python :

```
EVT.set_pv(vision["etat"], vision["present"], vision["delta_x"], vision["delta_z"], vision["distance"],
vision["r_yaw"], vision["r_z"])

match EVT.get_pv().get_state():
    case EtatVision.RECHERCHE:
        AXIS[Y].set_reference(0)
        AXIS[Z].CONTROLLING = True
        AXIS[Z].set_reference(math.floor(EVT.get_pv().get_z_reference()))
    case EtatVision.MISE_EN_AXE:
        AXIS[X].CONTROLLING = True
        AXIS[X].set_reference(0)
        AXIS[Y].set_reference(50)
        AXIS[Z].CONTROLLING = True
        AXIS[Z].set_reference(math.floor(AXIS[Z].get_nth_value(0) + 100 * EVT.get_pv().get_delta_z()))
    case EtatVision.TRAVERSEE:
        AXIS[Y].set_reference(50)
        AXIS[Z].CONTROLLING = False
        AXIS[Z].set_reference(0)
    case EtatVision.PERDU:
        AXIS[Y].set_reference(0)
    case _:
        AXIS[X].CONTROLLING = False
            AXIS[YAW].set_reference(math.floor(EVT.get_pv().get_yaw_reference()))
```

Des commandes sont également émises en X et en Y. L'axe Y est commandé en boucle ouverte avec une temporisation ; l'axe X a nécessité une approche spécifique.

Asservir l'axe x face aux perturbations

Pour corriger la trajectoire du drone sur l'axe droite / gauche, les mesures de vitesse du drone sont inexploitables (voir plus haut). Nous avons donc bouclé la commande sur la mesure visuelle.

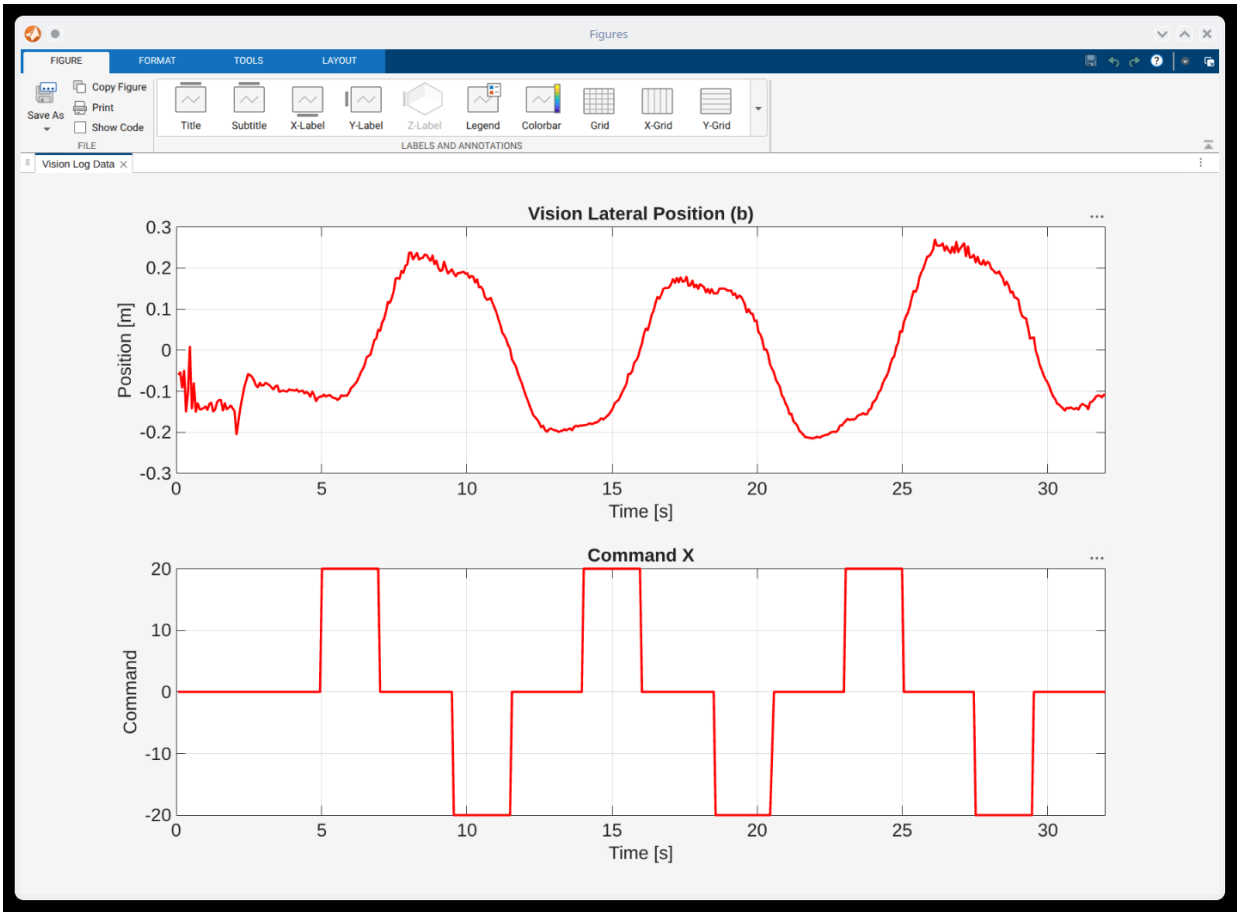
Plutôt qu'un modèle physique, nous identifions un modèle empirique entre la commande en pourcentage de vitesse et l'écart entre le centre de l'image et le centre du cerceau détecté. Le cerceau étant suffisamment large, une grande précision n'est pas nécessaire.



Avec donc :

- u(t) la commande en pourcent de vitesse sur l'axe droite / gauche.
- y(t) la sortie en pixel d'écarts entre le centre image et le centre cerceau.

Lors de l'expérience, le cerceau est tenu à la hauteur du drone, asservi en hauteur et à lacet nul. Des échelons successifs, positifs puis négatifs, sont envoyés. Les données obtenues sont bien plus exploitables que celles des accéléromètres :



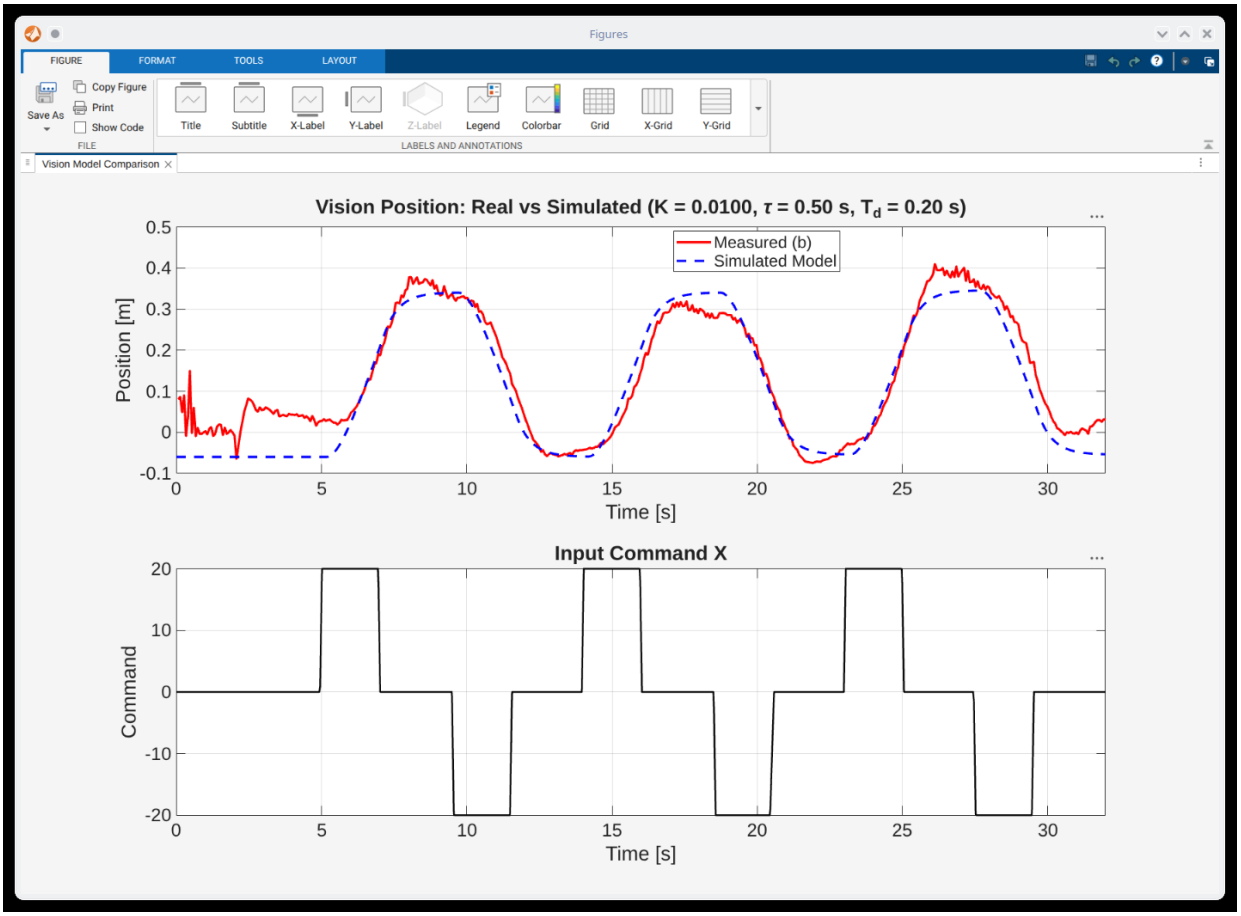
On estime un modèle du premier ordre avec intégrateur par la même méthode que pour la hauteur et le lacet :

$$G(s) = \frac{K}{s(1+\tau s)} e^{-T_d s}$$

Avec les valeurs :

- K = 0.01, le gain statique
- τ = 0.5, la constante de temps
- T_d = 0.2, le retard du système

Le modèle reproduit fidèlement les mesures :



En suivant la même méthodologie, on trouve pour un correcteur PD les gains suivants :

```
K_p = 0.262,
K_d = 0.8663,
```

La commande obtenue est satisfaisante. L'acquisition de l'écart, plus lente que la période d'échantillonnage du HIL (45 ms > 15 ms), produit toutefois des commandes impulsionnelles. Le résultat reste largement suffisant pour la traversée.

Une piste d'amélioration serait de passer à une représentation d'état et à une commande LQR.

5. Conclusion

Le drone détecte le cerceau, s'aligne sur lui et le traverse de manière autonome. Le projet a mobilisé l'ensemble de notre formation : identification et asservissement, vision par ordinateur et apprentissage profond, géométrie projective, informatique temps réel et réseau. Prévu pour un volume de 24 heures, il a représenté une soixantaine d'heures de travail à deux.

La principale difficulté a porté sur l'intégration : faire coopérer, sous contrainte temps réel, des traitements tournant à des fréquences différentes sur deux machines. Les pistes d'amélioration identifiées sont une commande par représentation d'état (LQR) et l'enrichissement du jeu de données avec d'autres cerceaux, filmés de loin et tenus à la main.